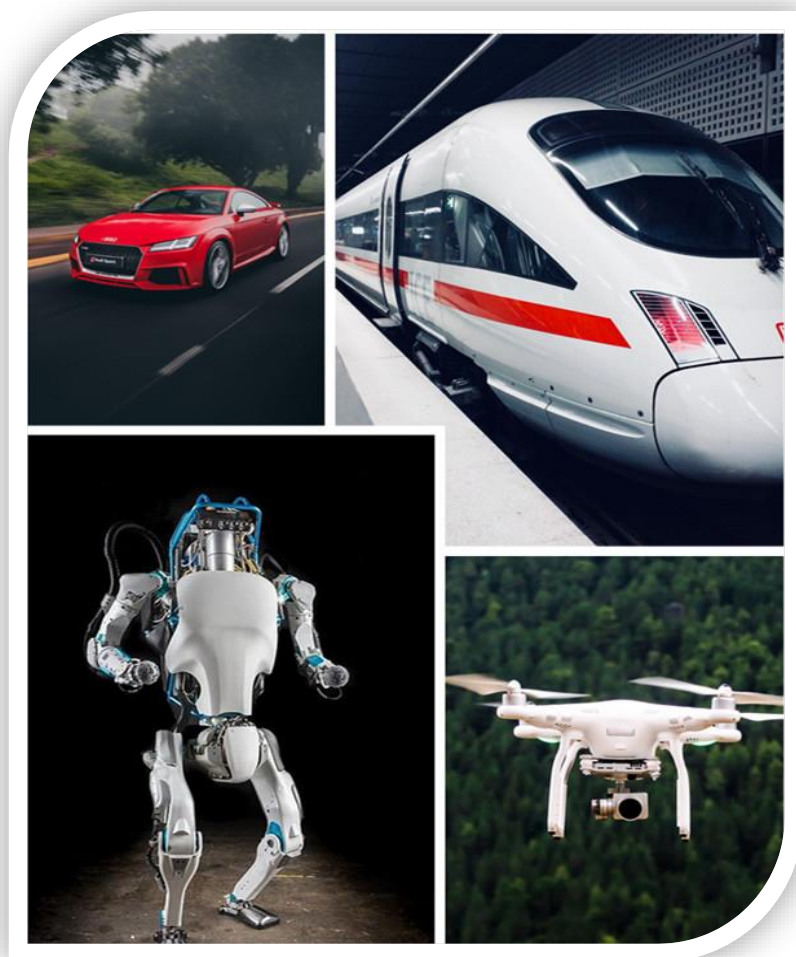


零基础入门

无刷直流电机矢量控制技术

第一版(发行版)

上官致远 ◆ 著



天津 xx 出版社

零基础入门 无刷直流电机矢量控制技术

第一版(发行版)

上官致远 ◆ 著

天津 xx 出版社

图字：02-2020-XXXX 号

内 容 简 介

本书共分八章，以无刷直流电机矢量控制技术为研究对象，从电机旋转的原理入手，分析无刷直流电机的原理，深入讲解矢量控制技术的原理和实现。对于矢量控制的关键技术环节做了进一步探讨，以STM32F4XX为平台，分析如何在该处理器上实现直流无刷电机的矢量控制算法。

研究了有感技术的实现，包括霍尔与编码器技术的分析，阐述了矢量控制的整个核心流程，对无刷直流电机的硬件驱动电路设计做了详细分析，并且介绍了开发过程中常见的问题和解决方法。

本书既可用于企业工程技术人员，也可用于高校学生以及教师等开发人员。

图书在版编目（CIP）数据

零基础入门无刷直流电机矢量控制技术/ 上官致远

天津：天津xx出版社，2020.2

ISBN 978-7—03-xxxxx-x

I .无... II .上官... III.零基础入门-无刷-直流-电机-矢量控制-技术 IV.TM345

中国版本图书馆CIP数据核字（2020 ）第XXXXXX号

策划编辑：xxx 责任编辑：xxx

责任印制：xxx 责任校对：xxx 责任发行：xxx

出版发行：天津xx出版社

社址：天津市南开区卫津路 邮政编码：300000

电话：022-2653xxxx转2028（发行部） 022-2653xxxx转321（总编室）

022-2653xxxx（读者服务部） 022-2653xxxx转305（质检部）

http: //www.tjxxpress.com.cn

经销新华书店

印刷：天津xx印务有限公司

开本：787x1092 1/16 版次：2020年2月第1版

印次 2020年2月第1次印刷

字数：300千字 定价 128.00元

版权所有·侵权必究·印装差错·负责调换

前言

几年前，一个偶然机会加入一家机器人公司，负责机器人的导航与控制算法研究。很多机器人底层的运动控制产品，价格不菲，且自主性差，于是萌生了自己开发伺服驱动的想法。于是不可避免的会涉及到无刷直流电机的驱动控制算法，特别是矢量控制算法。

无刷直流电机的矢量控制技术有着几十年的发展历史，并广泛应用于我们的日常生活中，比如新能源汽车、滚筒洗衣机、吸尘器、扫地机器人、电风扇等等；工业领域也有很多矢量控制技术的应用，常见的工业机器人、军用无人机等；可以说只要有电机的地方，几乎都会涉及到矢量控制技术的应用。

近几年随着机器人行业的火爆，矢量控制技术又一次引起了人们的关注。

矢量控制比传统的方波控制使得电机的转矩更为平稳，使用传统的方波控制技术，很难满足这些高精度、快速且平稳运行的场合需求。于是矢量控制技术成了这些高精度应用场合中电机控制的必备技术。

初次接触矢量控制技术，看一遍原理，感觉懂了，过几天再看，感觉又有很多不理解的地方，就这样在懂与不懂之间徘徊数月。

关于矢量控制技术线上线下的资料很多，但都很分散，很少有一本书能够把矢量控制的全部内容讲解透彻。而且矢量控制技术有一定的门槛，在开发过程中深知入门时的痛苦。为了给初学矢量控制的小伙伴一个指引，也给那些想在矢量控制方面继续深入研究的读者一些

帮助，决心写一本入门的书。

本书历经多个版本的打磨、修改、勘误，先后得到很多同行的指点和建议，最终形成现在的版本。当然由于作者能力有限，书中错误在所难免，欢迎大家交流指正，可在公众号后台获取联系方式。

上官致远

2020 年 2 月于天津

关于作者

毕业于天津大学，十余年嵌入式软件研发经验，服务欧美大型跨国企业多年，常年混迹米国，说着一口蹩脚的英语；做过平衡车、开发过无人机、玩过 Linux、折腾过 ROS，目前自主创业，从事 AGV 以及无刷电机驱动等相关技术的研发和推广工作。

平时忙于工作，偶尔写写公众号，主要分享矢量控制的相关技术，偶尔穿插一些关于职场，关于个人成长和认知提高的文章，感兴趣的可以关注一下「上官致远」。

为了普及无刷电机矢量控制的相关知识，我们有矢量控制的技术交流群，也建立了「无刷电机矢量控制」的知识星球，不定期举办一些线上线下的技术培训与交流，欢迎大家的加入。

聚焦个人成长，提高职场认知



「上官致远」

一个有料、有温度、更有态度的公众号

目录

第一章 电机基本原理	10
1.1 左手定则	10
1.2 有刷电机原理	11
1.3 无刷电机原理	12
第二章 方波驱动技术原理	14
2.1 霍尔传感器与电机换向原理	14
2.2 方波驱动技术原理	15
2.3 方波驱动技术的缺点	17
第三章 矢量控制技术原理	19
3.1 矢量控制技术概述	19
3.2 矢量控制技术原理详解	21
3.3 合成矢量的推导及 clarke 变换	23
3.4 park 变换	27
3.5 矢量控制架构	29
3.6 电流检测方式	31
3.7 坐标反变换	32
3.8 SVPWM 空间矢量脉宽调制	33
3.9 矢量所处扇区的判断	38
3.10 矢量作用时间分析	40
3.11 七段式 PWM 调制方式分析	47
3.12 标幺化与 Q15 格式	52

3.13 电流环与速度环	53
第四章 传感器与测速原理	55
4.1 编码器	55
4.2 M 法测速	56
4.3 T 法测速	58
4.4 M/T 法测速	59
4.5 霍尔区间位置估算	61
第五章 电机关键参数的测量	64
5.1 相电阻	64
5.2 相电感	64
5.3 反电动势常数	65
5.4 极对数	65
5.5 转动惯量	66
第六章 硬件电路设计与实现	68
6.1 电源电路设计	68
6.2 电流采集及其放大电路设计	70
6.3 逆变器驱动电路设计	71
6.4 过流、过压保护电路设计	72
6.5 MCU 处理器电路设计	73
6.6 通讯接口设计	74
6.7 PCB 设计注意事项	74
第七章 程序设计与调试流程	75

7.1 程序流程与架构	75
7.2 系统时钟及总线模块配置	77
7.3 PWM 模块配置	80
7.4 ADC 模块配置	84
7.5 霍尔接口配置	86
7.6 编码器接口配置	88
7.7 相电流采集与重构	91
7.8 FOC 核心算法	96
7.9 PI 调整方法	99
7.10 有感启动与调速实现	102
7.11 矢量控制调试流程	103
第八章 常见问题及解决方法	106
8.1 为什么看不到正弦波	106
8.2 速度波动为什么那么大	107
8.3 如何选择电机的最大转矩	107
8.4 电机运行时为什么会发烫	108
8.5 如何判断电机的噪声来源	108
参考文献	110

第一章 电机基本原理

1.1 左手定则

左手定则目的是为了判断通电导体在磁场中受力的方向，由英国电机学工程师弗莱明提出。

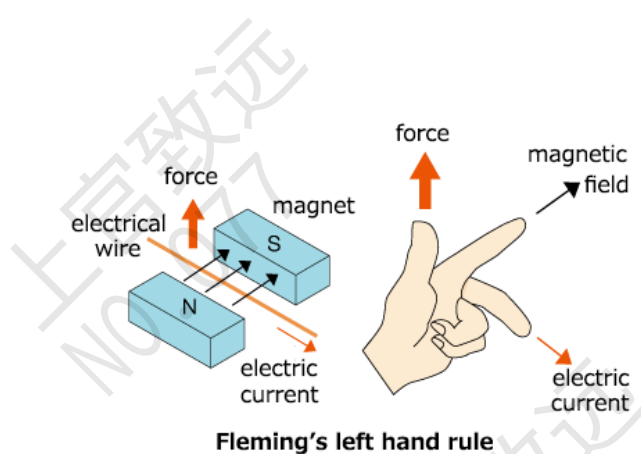


图 1-1 左手定则

左手三个手指呈相互垂直方式，食指指向磁场的方向，中指指向电流的方向，那么拇指指向就是导体受力的方向。

导体所受的力为： $F = B \times I \times L$

F 的单位为 N ， B 为磁通密度，单位为 Wb/m^2

I 为电流，单位为 A 。

L 为处于磁场中导体的长度，单位为 m 。

1.2 有刷电机原理

有刷直流电机通过换向器切换电流，以便于形成旋转的磁场，转子在定子磁场力的作用下旋转。

有刷直流电机由于采用电刷换向，因此，容易磨损，寿命短，接触点易产生灰尘，且噪音大。

当然，有刷电机也具备一定的优势，结构简单、转矩大，它的励磁方向始终垂直于磁场方向，控制方法简单有效。

另外，直流有刷电机具有很好的调速性能。

图 1-2 为直流有刷电机的原理示意图，电流通过换向电刷，适时的换向，形成磁场，与定子磁场相互作用，带动转子旋转。

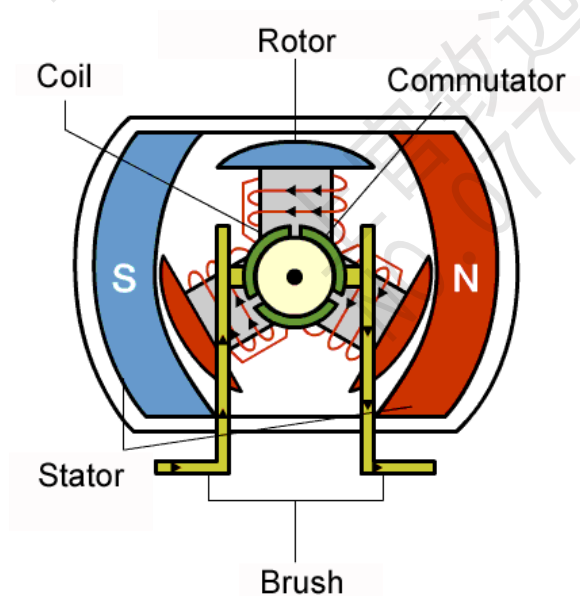


图 1-2 有刷电机

1.3 无刷电机原理

无刷直流电机通过电力电子器件代替碳刷，实时检测指示转子位置的霍尔元件，根据霍尔元件的状态控制电力电子器件的通断，进行有效的换向。

当然，也有很多无刷电机没有安装检测位置的传感器，而是依赖电机的运动关系和磁链方程，采用观测器估算转子位置，进行有效的控制。

无刷直流电机一般设置为三相，也有的小家电为了节省成本，只有两相。

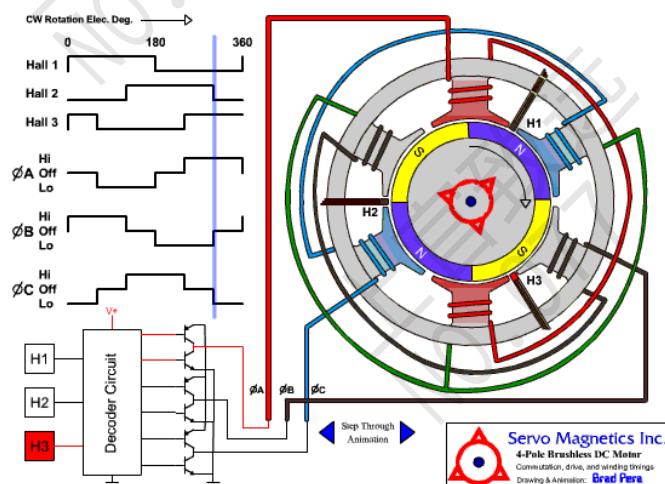


图 1-3 无刷直流电机

典型的无刷直流电机的逆变器驱动电路如下图，通过控制功率管的通断，切换电机的控制电流和方向。

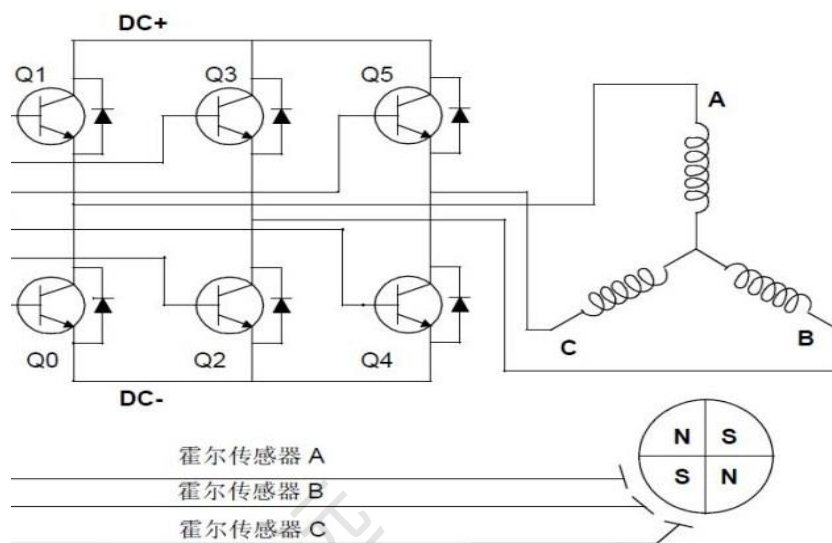


图 1-4 无刷直流电机逆变器驱动电路

无刷直流电机使用电力电子器件代替碳刷，虽然控制成本比有刷电机高，但是无刷电机的寿命长、噪音小、而且干净无粉尘，过载能力强，调速范围广，线性度好，应用也越来越广泛。

第二章 方波驱动技术原理

2.1 霍尔传感器与电机换向原理

开关型霍尔传感器为磁场敏感元件，当南北磁极通过时，它会呈现高低电平的变化，将霍尔元件严格按照定子换向边放置，当定子换向时，霍尔输出电平高低变化，这样就可以检测到定子的换向。

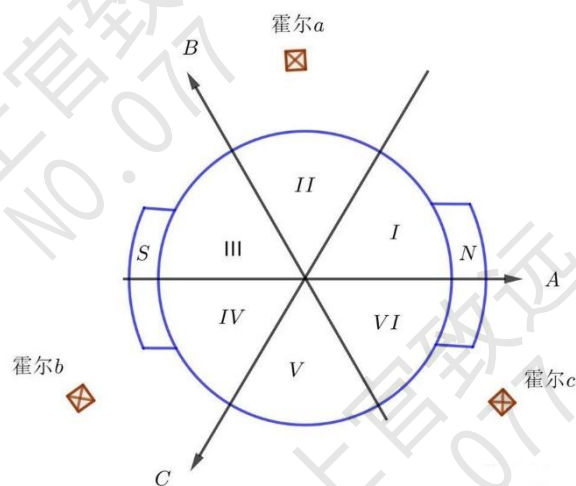


图 2-1 一对极电机霍尔安装位置

每个电机只需要三个霍尔，并呈 120° 电角度摆放，就能准确的检测到定子三相的换向时刻，也有厂家按 60° 电角度摆放，只是霍尔输出顺序不同，原理上一样。

每当一个磁极通过霍尔元件时，霍尔状态会改变一次，这样当磁场旋转一个电气周期，每相霍尔会改变两次状态，三路霍尔共改变六次状态。

电气角度指的是磁场旋转角度：机械角度 $\times$ 极对数 = 电气角度，电周期指的是磁场旋转的周期。

2.2 方波驱动技术原理

方波驱动，每次只给其中两相通电，第三相浮空，电流从电源的正极流进，进入其中一相，并从电机的另一相流出到电源的地。

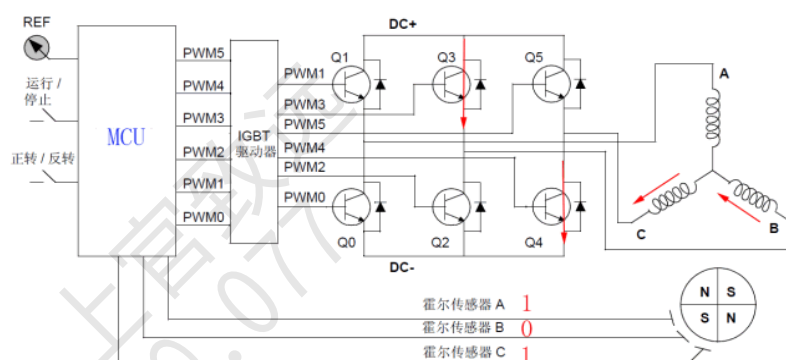


图 2-2 MCU 与电机逆变器驱动电路

下表是逆时针旋转的霍尔状态与各相导通顺序，顺时针旋转时，霍尔顺序相反，5 $\rightarrow$ 1 $\rightarrow$ 3 $\rightarrow$ 2 $\rightarrow$ 6 $\rightarrow$ 4。

霍尔A	霍尔B	霍尔C	打开的MOS管	UA	UB	UC
1	0	1	Q3 Q4	X	1	0
0	0	1	Q3 Q0	0	1	X
0	1	1	Q5 Q0	0	X	1
0	1	0	Q5 Q2	X	0	1
1	1	0	Q1 Q2	1	0	X
1	0	0	Q1 Q4	1	X	0

表 2-1 霍尔状态与换向顺序

转子旋转时，霍尔传感器会随着转子磁极而改变电平输出，因为

三相间隔 120° 电角度摆放, 因此三个霍尔的输出波形也是互相之间间隔 120° 电角度。

每个霍尔的状态会每隔 180° 改变一次, 三相霍尔将整个平面划分为 6 个扇区, 每隔 60° 电角度, 三个霍尔的组合值会改变一次。

根据霍尔的状态, 不断的改变三相通电顺序, 目的是形成旋转的磁场, 电机就会转动起来。

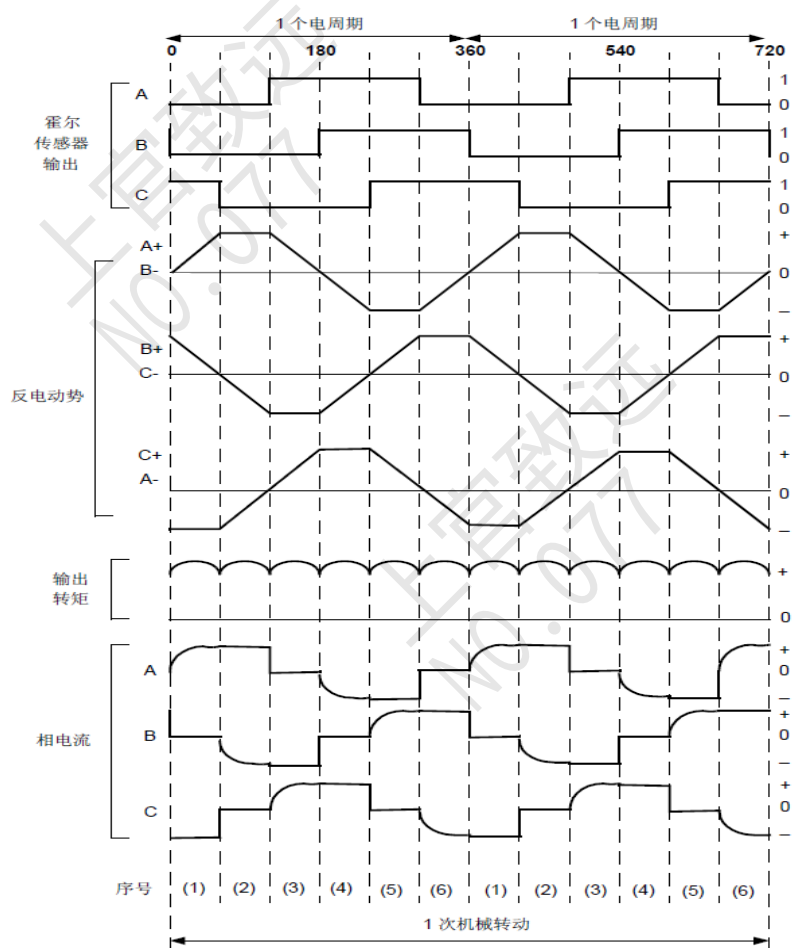


图 2-3 霍尔状态与反电动势和相电流的关系

直流电机的转速和电压是成正比的, 如果想改变转速只需要改变导通的 MOS 管的 PWM 占空比即可, 相当于调压。

常见的 PWM 的调节方式有三种：

a. H_PWM/L_ON, 即下桥臂打开, 只调节上桥臂。比如第一个状态, U_b 给正 U_c 给负, Q3 Q4 打开, 电流从 Q3 流向 b 再经 c 流出到 Q4 最后至电源的负极。那么就可以只调节上桥臂 Q3 的占空比, 而 Q4 一直打开。

b. H_ON/L_PWM 即上桥臂一直打开, 只调节下桥臂, 比如 Q3 常开只调节 Q4。

c. H_PWM/L_PWM 即互补调节, Q3 和 Q2 为一组互补输出, Q5 与 Q4 为一组互补输出, Q1 和 Q0 关断; 互补的好处是上管关断的瞬间电流可以由下管续流而不经 MOS 管的内部续流二极管, 以免在大电流时造成 MOS 管的烧坏。

PWM 的频率一般选择在 20K 左右, 一是考虑到 MOS 开关管的承受能力, 另外一个尽量避免高次谐波的干扰。

2.3 方波驱动技术的缺点

a. 方波驱动时, 电机相电流只有通和断两种状态, 即使占空比很小, 平均电流很小的状态下, 线圈电流的脉动峰值电流也会很大, 由于发热量与电流的平方成正比, 因此方波驱动时系统发热量大, 损耗多。

b. 方波控制时, 磁场不是连续的, 是跳跃的, 因此会造成转矩的脉动, 在对转矩要求较高的场合, 方波驱动是不合适的, 它会造成系

统的不稳定，控制精度会相应的降低。

c. 由于转矩的脉动会造成系统的震动和噪音，震动会干扰系统本身的稳定性和板载芯片或传感器性能下降。

当然方波驱动技术也有它的优点，控制算法简单，硬件成本低。目前市面上也有很多产品依然采用方波驱动技术。

第三章 矢量控制技术原理

3.1 矢量控制技术概述

据不完全统计，电机占据了全球将近一半的用电量。传统电机的控制方式已经无法满足新时代的要求。随着人们环保意识的日益增强，迫切需要更加节能、更加易控、效率更高的电机设备。

传统三相交流感应电机输入的是三相对称正弦电压，空间磁链近乎圆形，转矩稳定。但是它也有一定的缺点：

- 三相对称正弦交流电产生的是一个随时间和空间都在变化的旋转磁场，是一个多变量的系统；
- 转子磁势和定子磁通之间不存在垂直关系；
- 定子电流无法单独调节励磁和转矩，他们之间是强耦合，非线性的复杂关系；
- 体积大，损耗多；

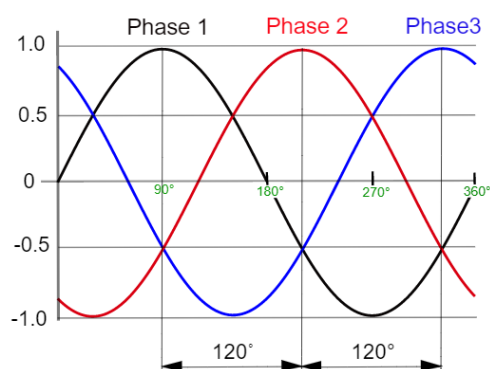


图 3-1 三相对称交流输入

于是人们想到，能不能将三相交流感应电机像直流电机那样控制起来简单有效，还特别稳定？

为了解决这个问题，德国达姆施塔特工业大学 (Darmstadt) 的 K.Hasse 和西门子公司 F.Blaschke，在上世纪七十年代创造性的提出了一种矢量控制方式，将三相交流信号经过一系列坐标变换，最终变为直流可控的两相正交电流。解耦复杂的电流关系，使得交流电机变得简单可控。

矢量控制技术既可用于控制交流电机，同样也适用于控制直流无刷电机。

无论是哪种电机，它的转矩都正比于定子磁场和转子磁场的叉乘，即他们所围成的平行四边形的面积。

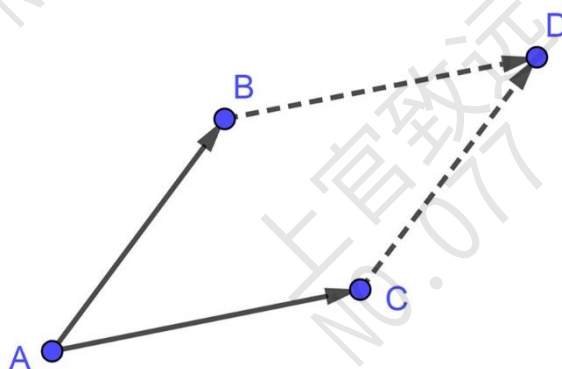


图 3-2 定子磁场与转子磁场的合成

$$\vec{T}_m = \vec{AB} \times \vec{AC}$$

很明显，当定子磁场和转子磁场夹角为 90° 的时候，他们所围成的平行四边形的面积最大，这个时候所产生的力矩也最大。

正如前面提到的直流有刷电机，它的定子励磁电流和电枢电流各自在自己的回路中，且分别可控，定子磁场和转子磁场始终能保持垂

直，所产生的力矩也最大。

如果想在控制上达到直流有刷电机的效果，那就必须想办法解耦转矩与励磁之间的关系。如果能控制定子磁场和转子磁场的夹角始终保持 90° ，那么无刷电机的控制效率会大大提高，这就是 FOC 矢量控制技术产生的背景。

3.2 矢量控制技术原理详解

在正式分析矢量控制技术之前，特别的介绍一下正弦波控制技术。它是一种处于方波与矢量控制之间的一种技术。

严格来说，它是一种 PWM 的调制方式，将方波 PWM 按正弦调制，虽然实现了定子正弦电流，但是它只能控制电流的大小，不能控制电流的方向。

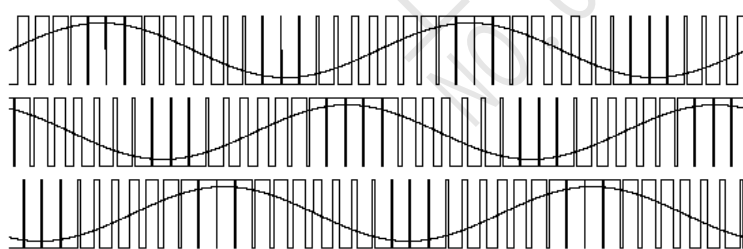


图 3-3 按正弦方式调制的 PWM

而矢量控制技术既能控制电流大小，也能控制电流方向。

矢量控制技术，英文称为 FOC (Field-oriented control)，严格来说，应该叫做磁场定向控制。它能够解耦复杂的定子电流关系，将

定子电流分解成控制励磁的直轴电流，以及控制转矩的交轴电流。

前面提到，三相电机通入的是空间相差 120° 的三路对称正弦电压，在空间形成旋转的磁场。当然，想要在空间产生旋转的磁场，不一定非要三相对称绕组，任意对称的多相绕组都可以在空间产生旋转的磁动势，特别是两相对称正交绕组，也可以达到同样的效果，且两相为相互垂直的独立变量。

因此，我们可以将三相电机的模型设想成两相模型。以产生和三相电机同样的圆形磁场为准则，两相在空间上互差 90° ，一个负责转矩 F 的控制，一个负责励磁 M 的控制，他们之间互不影响。

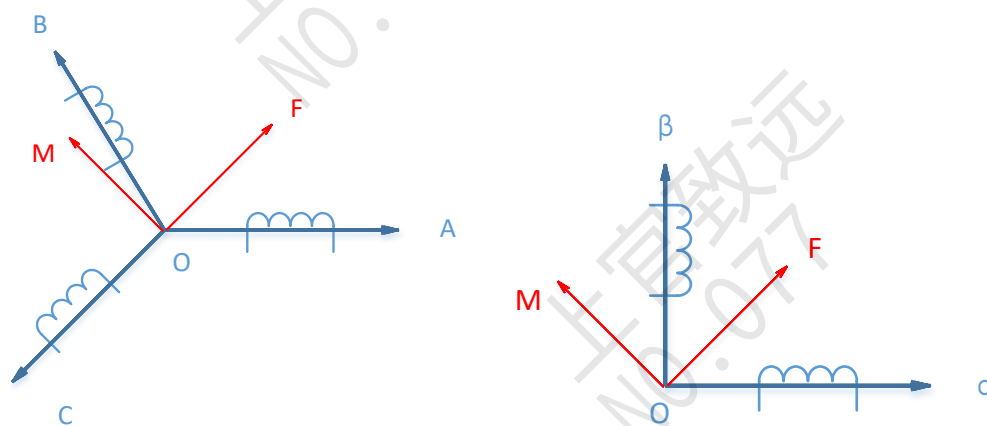


图 3-4 三相电机与两相电机的等效模型

三相绕组产生的磁场和力矩同两相正交绕组产生的磁场和力矩的大小、方向完全相等，并且以同样的角速度在空间逆时针旋转。这就是常说的三相静止坐标系到两相静止坐标系的变换。

再进一步，我们假定有一个两相正交的对称绕组，分别通以直流电流 i_d 、 i_q ，他们产生的合成磁动势和两相静止坐标系及三相静止

坐标系完全相等，且以磁场同样的角速度旋转，那么 d 、 q 旋转坐标系和前面的三相静止以及两相静止坐标系就可以完全等效，这就是两相静止坐标系到两相旋转坐标系的变换。

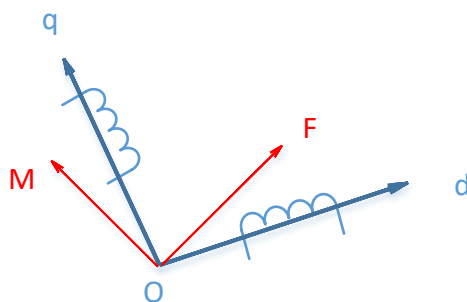


图 3-5 两相旋转坐标系

因此，三相静止坐标系下的 i_a 、 i_b 、 i_c 完全可以等效为两相静止坐标系下的 i_α 、 i_β 以及两相旋转坐标系下的 i_d 、 i_q 。

得到 i_d 、 i_q 之后，三相电机的多变量、强耦合、非线性的系统控制将直接变成两个相互独立的直流分量的控制，解耦了复杂的三相电机多变量的复杂关系，使系统控制变得简单。

接下来我们分析它们之间的变换关系。

3.3 合成矢量的推导及 clarke 变换

很多文档和资料中，对三相电机的合成矢量只是给出了结果，省去了中间推导过程。很多人看到这里，一脸茫然，不明白结果是怎么来的，其实推导过程并不复杂，下面以三相对称正弦电流合成矢量为例，进行详细的推导。

三相对称正弦电流可以这样表示：

$$\begin{cases} I_a = I_m \cos \omega t \\ I_b = I_m \cos \left(\omega t - \frac{2\pi}{3} \right) \\ I_c = I_m \cos \left(\omega t + \frac{2\pi}{3} \right) \end{cases}$$

其中 I_m 为幅值, $\omega = 2\pi f$ 为角速度, 三相电流在空间上互差 120° 。

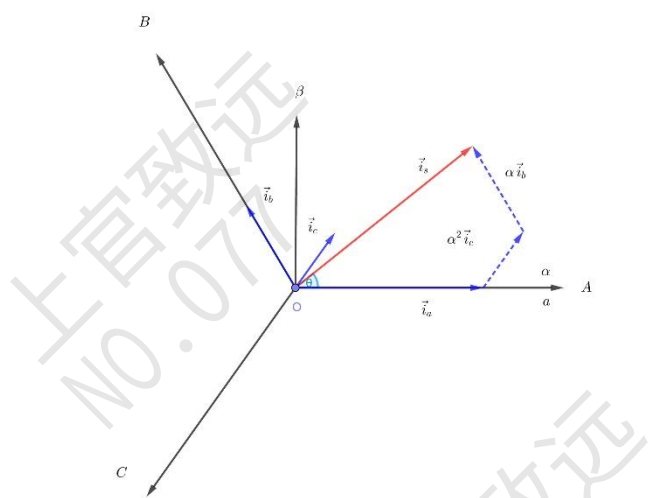


图 3-6 三相静止坐标与两相静止坐标

它们的合成矢量可以表示为 $\vec{I}_s = \vec{I}_a + \vec{I}_b + \vec{I}_c$

即:

$$\vec{I}_s = I_m \cos(\omega t) + I_m \cos\left(\omega t - \frac{2\pi}{3}\right) e^{j\frac{2\pi}{3}} + I_m \cos\left(\omega t + \frac{2\pi}{3}\right) e^{-j\frac{2\pi}{3}}$$

根据欧拉公式 $e^{jx} = \cos x + j\sin x$ 可以推出:

$$\begin{aligned} \vec{I}_s &= I_m \cos(\omega t) + I_m \cos\left(\omega t - \frac{2\pi}{3}\right) \left(\cos \frac{2\pi}{3} + j \sin \frac{2\pi}{3}\right) \\ &\quad + I_m \cos\left(\omega t + \frac{2\pi}{3}\right) \left(\cos \frac{2\pi}{3} - j \sin \frac{2\pi}{3}\right) \end{aligned}$$

进一步计算:

$$\begin{aligned}\vec{I}_s &= I_m \cos(\omega t) + I_m \left(\cos \omega t \cos \frac{2\pi}{3} + \sin \omega t \sin \frac{2\pi}{3} \right) \left(-\frac{1}{2} + j\frac{\sqrt{3}}{2} \right) \\ &\quad + I_m \left(\cos \omega t \cos \frac{2\pi}{3} - \sin \omega t \sin \frac{2\pi}{3} \right) \left(-\frac{1}{2} - j\frac{\sqrt{3}}{2} \right)\end{aligned}$$

代入三角函数值得出：

$$\begin{aligned}\vec{I}_s &= I_m \cos(\omega t) + I_m \left(-\frac{1}{2} \cos \omega t + \frac{\sqrt{3}}{2} \sin \omega t \right) \left(-\frac{1}{2} + j\frac{\sqrt{3}}{2} \right) \\ &\quad + I_m \left(-\frac{1}{2} \cos \omega t - \frac{\sqrt{3}}{2} \sin \omega t \right) \left(-\frac{1}{2} - j\frac{\sqrt{3}}{2} \right)\end{aligned}$$

进一步分解：

$$\begin{aligned}\vec{I}_s &= I_m \cos(\omega t) + I_m \left(\frac{1}{4} \cos \omega t - \frac{\sqrt{3}}{4} \sin \omega t - j \frac{\sqrt{3}}{4} \cos \omega t + j \frac{3}{4} \sin \omega t \right) \\ &\quad + I_m \left(\frac{1}{4} \cos \omega t + \frac{\sqrt{3}}{4} \sin \omega t + j \frac{\sqrt{3}}{4} \cos \omega t + j \frac{3}{4} \sin \omega t \right)\end{aligned}$$

最后得出：

$$\vec{I}_s = I_m \frac{3}{2} \cos \omega t + j \frac{3}{2} \sin \omega t$$

即：

$$\vec{I}_s = \frac{3}{2} I_m e^{j\omega t}$$

由此可以看出，空间上互差 120° 的三相对称正弦电流的合成矢量是一个角速度为 ω ，且绕中心点旋转的矢量，因此它能形成一个旋转的磁场。它的幅值是单相幅值的 $\frac{3}{2}$ 倍。

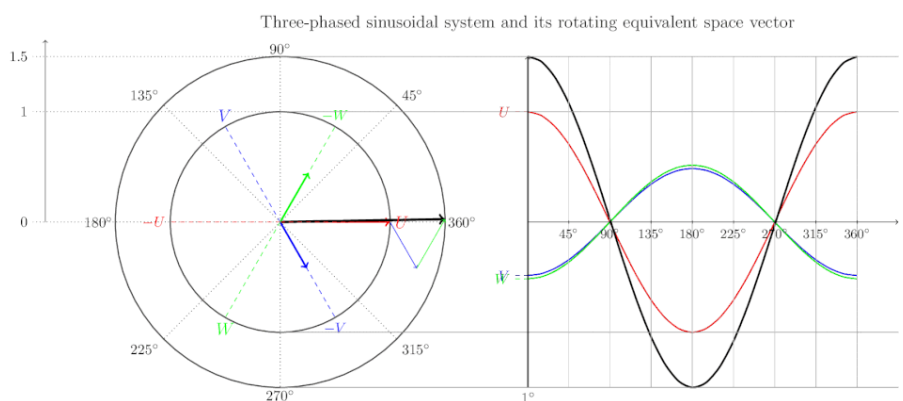


图 3-7 合成矢量效果图

从三相静止坐标系 A、B、C 到两相静止坐标系 α 、 β 的变换过程，就是我们通常所说的 clarke 变换，简称 3S/2S 变换。

接下来，我们从坐标投影对 clarke 变换做进一步分析。

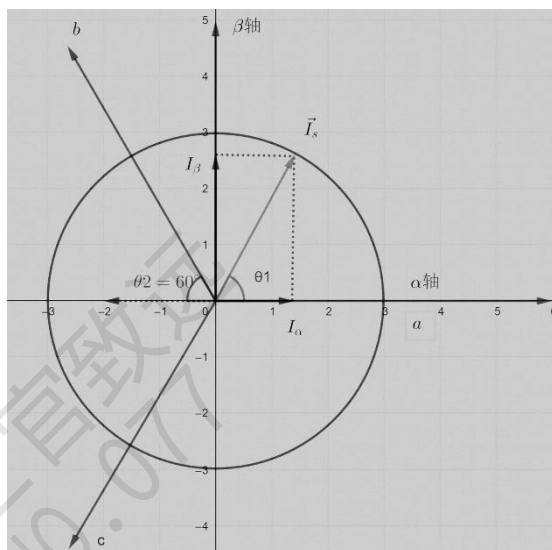


图 3-8 Clarke 变换

将三相对称电流矢量 I_a 、 I_b 、 I_c 分别在 α 、 β 轴投影，得到它们在两相坐标系下的分量：

$$\begin{cases} I_\alpha = I_a - I_b \cos \frac{\pi}{3} - I_c \cos \frac{\pi}{3} \\ I_\beta = I_b \sin \frac{\pi}{3} - I_c \sin \frac{\pi}{3} \end{cases}$$

将上式写成矩阵形式：

$$\begin{bmatrix} I_\alpha \\ I_\beta \end{bmatrix} = k \begin{bmatrix} 1 & -\frac{1}{2} & -\frac{1}{2} \\ 0 & \frac{\sqrt{3}}{2} & -\frac{\sqrt{3}}{2} \end{bmatrix} \begin{bmatrix} I_a \\ I_b \\ I_c \end{bmatrix}$$

假设变换前电流有效值为 I_m ，根据前面推导的三相合成公式，那

么变换后为 $\frac{3}{2}I_m$ 。对于电压同样适用，假设变换前电压有效值为 U_m ，变换后则为 $\frac{3}{2}U_m$ 。为了保证变换前后的幅值不变，即合成矢量的大小和方向相等，那么必须要在变换时乘以一个系数 k 。

很明显系数 $k = \frac{2}{3}$ 。

将系数 $k = \frac{2}{3}$ 代入上式得出：

$$\begin{bmatrix} I_\alpha \\ I_\beta \end{bmatrix} = \frac{2}{3} \begin{bmatrix} 1 & -\frac{1}{2} & -\frac{1}{2} \\ 0 & \frac{\sqrt{3}}{2} & -\frac{\sqrt{3}}{2} \end{bmatrix} \begin{bmatrix} I_a \\ I_b \\ I_c \end{bmatrix}$$

再根据基尔霍夫电流定律 $I_a + I_b + I_c = 0$ 得出等幅值 Clarke 变换的最终结果：

$$\begin{cases} I_\alpha = I_a \\ I_\beta = \frac{1}{\sqrt{3}}I_a + \frac{2}{\sqrt{3}}I_b \end{cases}$$

如果三相到两相为等功率变换，假设变换前的功率为 P_0 ，变换后的功率为 P_1 那么有：

$$P_0 = U_m \times I_m \times 3$$

$$P_1 = \frac{3}{2}U_m \times k \times \frac{3}{2}I_m \times k \times 2$$

令 $P_0 = P_1$ ，可以推出如果是等功率变换，系数 $k = \sqrt{\frac{2}{3}}$ 。

3.4 park 变换

矢量控制的目的是解耦复杂的电流关系，上面的 clarke 变换由三相静止坐标系到两相静止坐标系后，电流依然由时间和速度决定，因此还需要进一步变换。将 α 、 β 决定的两相静止坐标系变换为随磁场

旋转的 d、q 坐标系。

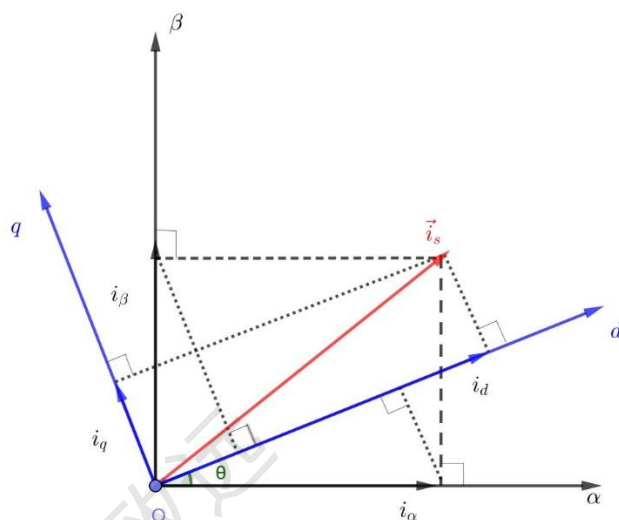


图 3-9 park 变换

i_α 、 i_β 为合成矢量 $\vec{I}_s$ 分别在 α 与 β 决定的两相静止坐标轴上的投影。坐标系 d q 为定子磁场旋转坐标系, q 轴垂直于磁场, 称为交轴; d 轴平行于磁场, 称为直轴。

角度 θ 是磁链与两相静止坐标系 α 轴的夹角。由坐标投影与三角关系, 可以推出:

$$\begin{cases} i_d = i_\alpha \cos \theta + i_\beta \sin \theta \\ i_q = -i_\alpha \sin \theta + i_\beta \cos \theta \end{cases}$$

下图形象的描述了 clarke 变换与 park 变换的电流波形

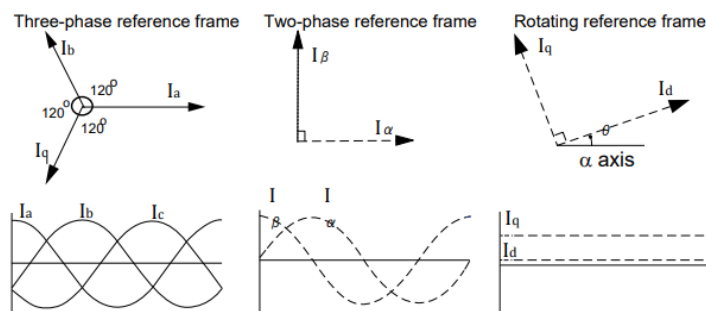


图 3-10 坐标变换波形

3.5 矢量控制架构

Park 变换之后，最终将三相强耦合、非线性，复杂的电流关系变换成随磁场旋转的 d、q 轴电流，可以分别控制转矩和励磁，达到了解耦的目的。接下来的问题是，如何实现矢量控制，最终达到控制电机的目的。

首先，控制电机最终是力的控制，也就是电流的控制，所以有控制就一定有反馈，有闭环，因此矢量控制里先有电流控制环。

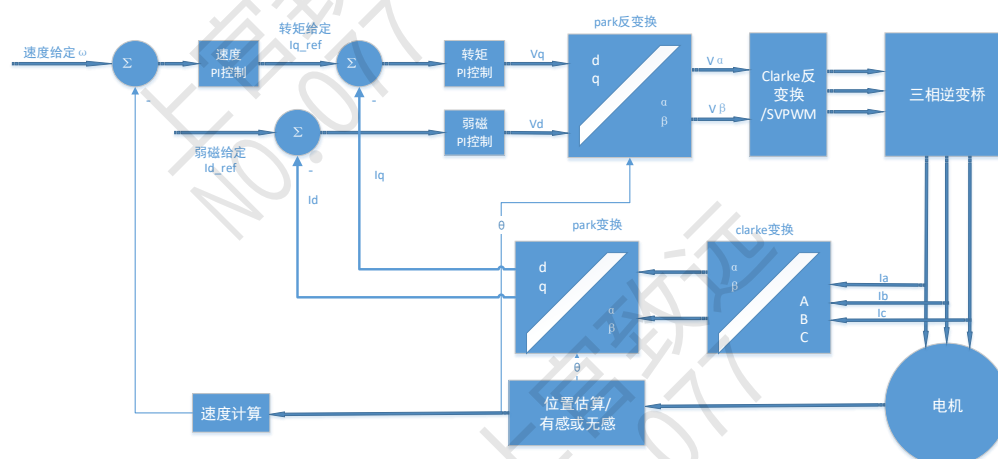


图 3-11 矢量控制架构

电流环外部是速度环，通过安装在电机底部的位置传感器或者无感观测器算法获得转子位置信息，并计算反馈速度，与给定的速度进行比较，送入速度 PI 控制器，并输出转矩电流；之后这个转矩电流会继续进入电流环转矩 PI 控制器，最终输出 d、q 轴电压，这个电压是无法直接驱动电机的，因此需要将 d、q 两相旋转坐标系下的电压经过 park 反变换以及 clarke 反变换，最终变换为 a、b、c 三相静止坐标系下的电压，最终才能驱动三相电机。

常见的矢量控制方式是让 d 轴电流为 0，控制 q 轴电流，因为 d 轴电流对转矩没有贡献。因此，可以最大化的利用电流，在相同转矩下，电流可以做到最小。当然还有其他几种矢量控制的方式，包括最大转矩控制以及弱磁控制等。特别是第一种方式较为常见，因此，本书采用的是让 d 轴电流为 0，控制 q 轴电流的矢量控制方式。

采用该方式的矢量控制整体流程可以总结为：

- a. 通过 ADC 测量分流电阻的电压值，并转换成三相电流值。根据基尔霍夫电流定律，同一时刻流入某一节点的电流等于流出该节点的电流，因此，同一时刻只需要采集三相电流中的两相即可重构出三相电流。
- b. 将采集到的电流值，通过 clarke 与 park 变换，最终得到 d、q 轴电流。
- c. 通过位置传感器或者角度估算器进行转子电气角度和转子角速度的计算。
- d. 根据目标速度与实际反馈速度进行比较，送入速度 PI 控制器进行速度控制，输出电流环需要的 I_q 、 I_d 。
- e. 根据速度环输出的电流值送入电流 PI 控制器最终输出需要的电压值 V_q 、 V_d 。
- f. 将 d、q 轴的电压通过坐标反变换以及 SVPWM，最终形成 PWM 占空比，控制逆变器输出。

实际使用中可能还会涉及到位置的控制，因此有时候还需要加入位置环，形成位置-速度-电流，三闭环控制结构。

3.6 电流检测方式

常见的电流检测方式分三种：

- a. 三电阻方式：程序设计相比单电阻要简单，将三个电阻分别连接至逆变器三个桥臂的下端，在下桥臂打开时进行采样，每次只检测其中两路即可。电压以地电压为中心，正负摆动，因此需要相移电路，并送至运算放大器进行放大。
- b. 单电阻方式：程序相对复杂，只在电源的地端放置一个检测电阻，在特定的时刻采集电流。检测到的电压只有正值，不需要相移电路，只需要一路运算放大器放大处理，节省成本。
- c. 双电流互感芯片方式：抗干扰能力强，但成本高。任何时刻都可以采集电流。同样，只需要安装两路即可，通过基尔霍夫电流定律，计算出第三相的电流。

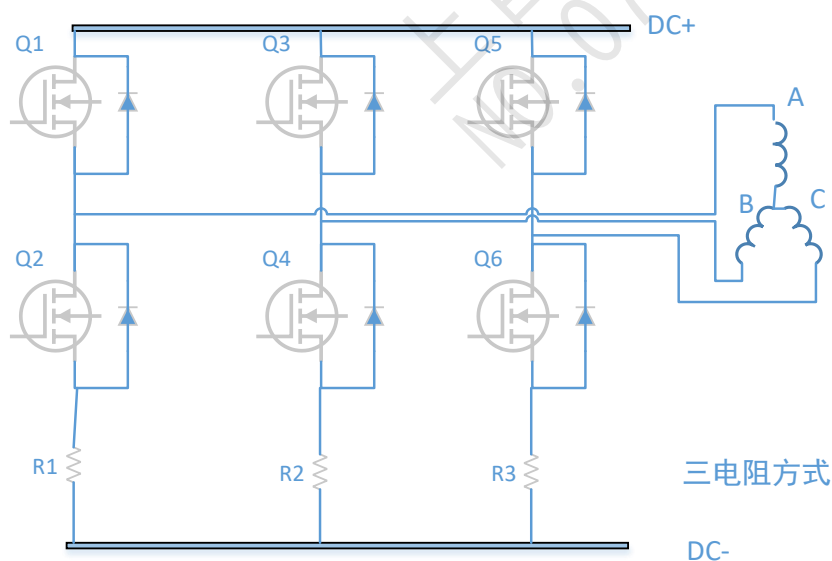


图 3-12 三电阻电流检测方式

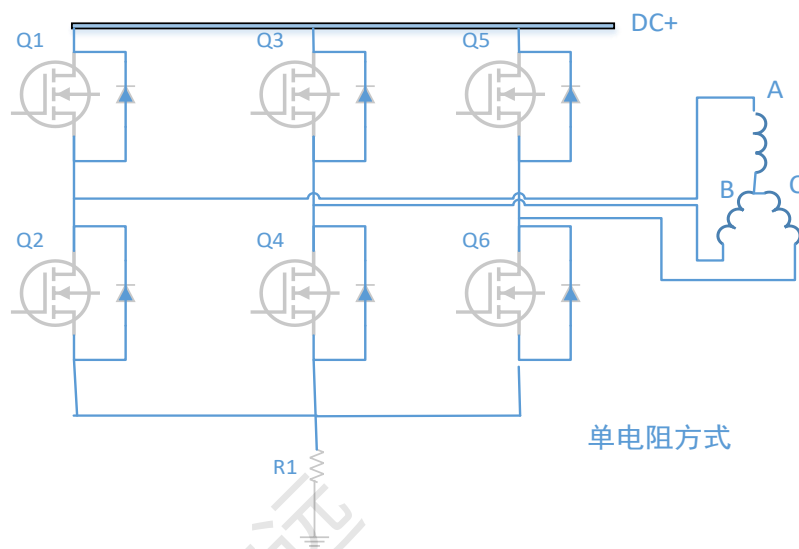


图 3-13 单电阻电流采集方式

3.7 坐标反变换

在矢量控制的最后一个环节需要将电流环输出的 V_q 、 V_d 经过坐标反变换形成三相电机所需要的电压值，才能送到电机的 u 、 v 、 w 三相中去。

这是一个逆变换的过程，称为 park 反变换，将两相旋转坐标系下的 V_q 、 V_d 变换成 α 、 β 两相静止坐标系的 V_α 、 V_β 。

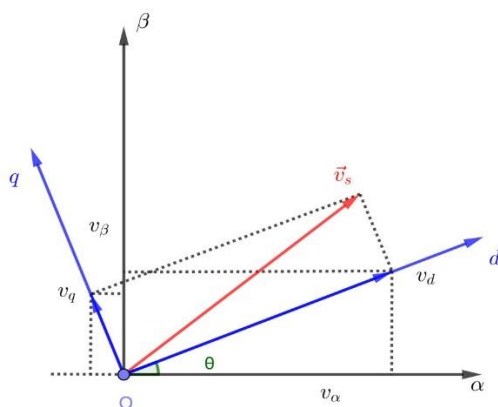


图 3-14 park 反变换

将 V_q 、 V_d 分别投影到 α 、 β 轴，由三角关系可以推出：

$$\begin{cases} v_\alpha = v_d \cos \theta - v_q \sin \theta \\ v_\beta = v_d \sin \theta + v_q \cos \theta \end{cases}$$

接下来是通过两相静止坐标系下的 v_α 、 v_β 获得能够直接驱动三相电机的 a、b、c 三相静止坐标系下的电压，这一步骤通过 SVPWM 实现。

3.8 SVPWM 空间矢量脉宽调制

矢量控制的目的是解耦复杂的定子电流关系，使三相交流电机的控制能像直流电机一样，控制起来简单有效，且必须保证矢量变换前后，系统所产生的旋转磁场等效。

在 park 反变换后，得到的是两相静止坐标系下的 v_α 与 v_β ，接下来的问题是如何利用 v_α 与 v_β 控制功率管的通断，产生和之前一样的旋转等效磁场。

前面分析到，三相交流电机通入三路对称且相互之间相差 120° 的正弦电流就可以在空间形成旋转的磁场。

但是，无刷电机是依靠六个功率管的开关组合来控制，不可能通入三相对称的正弦电流，因此只有想办法通过三相逆变器的功率管的不同组合来模拟生成需要的旋转矢量。

SVPWM 的全称是 Space Vector Pulse Width Modulation 即空间矢量脉宽调制。它通过三相逆变器的六个 MOS 管输出随时间改变的 PWM 调制波，模拟三相对称正弦电流所形成的圆形矢量，产生接近圆形的磁链轨迹。

SVPWM 利用平均值等效原理，在一个开关周期内，对基础矢量加以

组合，使其产生的效果与期望电压矢量相同，根据不同的电压矢量，判断其所在的扇区区间，利用相应扇区的基础矢量不断的合成需要的电压矢量，这样当电压矢量转动一圈之后，即在空间形成一个旋转的近似圆形的磁场。下图为典型的三相逆变器驱动电路。

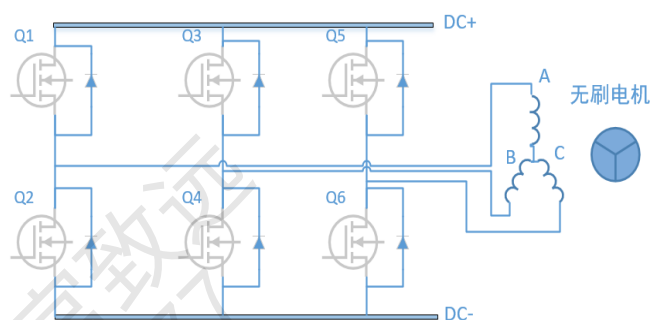


图 3-15 三相逆变器驱动电路

Q1~Q6 为三相逆变器的六个功率开关管，同一时刻，同一桥臂上下两个功率管不能同时导通，否则会造成电源短路，烧坏管子。

因此一般会在互补 PWM 波中加入死区，防止短路造成的危害。

由于上下管子必须互补操作，因此可以使用上桥臂的管子的状态表示该桥臂对应的那一相的状态，因为只有当上桥臂打开后，对应的那一相才有正向电流流过，反之为负向电流。

比如 Q1 打开，Q2 截止，那么电流会从 DC+ 通过 Q1 流入电机的 A 相，如果 Q1 截止，Q2 打开，电流会从电机的 A 相流出经 Q2 到电源的 DC-。

假定，上桥臂打开，下桥臂截止，该桥臂的状态为 1，反之为 0；因为有三个桥臂，因此我们可以定义开关函数 $S_x(x = a, b, c)$ 为：

$$S_x = \begin{cases} 1 & \text{上桥臂打开, 下桥臂关闭} \\ 0 & \text{上桥臂关闭, 下桥臂打开} \end{cases}$$

于是三个桥臂的状态可以有 $2^3 = 8$ 种, 也就是 ABC 三相共有 8 种状态组合。

假设 A 相上桥臂导通, 下桥臂关闭, B 相上桥臂关闭, 下桥臂导通, C 相上桥臂关闭, 下桥臂导通, 也就是 $S_a=1, S_b=0, S_c=0$, 此时的等效电路为:

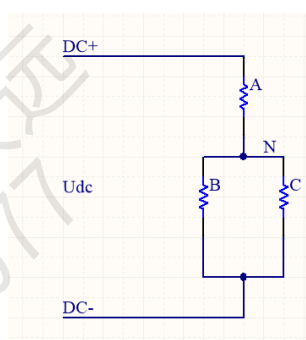


图 3-16 状态 $U_0(100)$ 时的等效电路

我们用 U_a 、 U_b 、 U_c 表示各相的相电压, 相电压表示各相相对于电机中性点 N 的电压:

$$\begin{cases} U_a = U_A - U_N \\ U_b = U_B - U_N \\ U_c = U_C - U_N \end{cases}$$

由上面的等效电路, 可以推出:

$$\begin{cases} U_a = \frac{2}{3} U_{dc} \\ U_b = -\frac{1}{3} U_{dc} \\ U_c = -\frac{1}{3} U_{dc} \end{cases}$$

将上面计算的值, 代入三相对称正弦电压矢量合成的公式:

$$U_{out} = \frac{2}{3} \left(U_a + U_b e^{j\frac{2\pi}{3}} + U_c e^{-j\frac{2\pi}{3}} \right)$$

前面的系数 $\frac{2}{3}$ 是为了保证变化前后幅值不变原则，因此可以得出：

$$U_{out} = \frac{2}{3} \left(\frac{2}{3} U_{dc} - \frac{1}{3} U_{dc} \left(\cos \frac{2\pi}{3} + j \sin \frac{2\pi}{3} \right) - \frac{1}{3} U_{dc} \left(\cos \frac{2\pi}{3} - j \sin \frac{2\pi}{3} \right) \right)$$

整理后得：

$$U_{out} = \frac{2}{3} U_{dc} = \frac{2}{3} U_{dc} e^{j0}$$

同理可以推出其他几个状态下的相电压，并根据三相合成的公式，可以推出合成矢量的大小和方向。

向量	S_1	S_2	S_3	A	B	C	U_a	U_b	U_c	U_{out}
$U_z(000)$	0	0	0	0	0	0	0	0	0	0
$U_0(100)$	1	0	0	1	0	0	$\frac{2}{3}U_{dc}$	$-\frac{1}{3}U_{dc}$	$-\frac{1}{3}U_{dc}$	$\frac{2}{3}U_{dc}$
$U_{60}(110)$	1	1	0	1	1	0	$\frac{1}{3}U_{dc}$	$\frac{1}{3}U_{dc}$	$-\frac{2}{3}U_{dc}$	$\frac{2}{3}U_{dc}e^{j\frac{\pi}{3}}$
$U_{120}(010)$	0	1	0	0	1	0	$-\frac{1}{3}U_{dc}$	$\frac{2}{3}U_{dc}$	$-\frac{1}{3}U_{dc}$	$\frac{2}{3}U_{dc}e^{j\frac{2\pi}{3}}$
$U_{180}(011)$	0	1	1	0	1	1	$-\frac{2}{3}U_{dc}$	$\frac{1}{3}U_{dc}$	$\frac{1}{3}U_{dc}$	$\frac{2}{3}U_{dc}e^{j\pi}$
$U_{240}(001)$	0	0	1	0	0	1	$-\frac{1}{3}U_{dc}$	$-\frac{1}{3}U_{dc}$	$\frac{2}{3}U_{dc}$	$\frac{2}{3}U_{dc}e^{j\frac{4\pi}{3}}$
$U_{300}(101)$	1	0	1	1	0	1	$\frac{1}{3}U_{dc}$	$-\frac{2}{3}U_{dc}$	$\frac{1}{3}U_{dc}$	$\frac{2}{3}U_{dc}e^{j\frac{5\pi}{3}}$
$U_z(111)$	1	1	1	1	1	1	0	0	0	0

表 3-1 开关组合与基础矢量

从上面的计算可以看出，合成矢量可以表示成： $U_{out} = \frac{2}{3} U_{dc} e^{j\omega t}$ ， $\omega = 2\pi f$ ，可以看出六个基础矢量的模长等值，且为 $\frac{2}{3} U_{dc}$ 。

将上面的 8 个矢量放到矢量分布图中，红色为 A、B、C 三相空间静止坐标系， $U_0(100)$ 表示只有 A 相导通，所以它与 A 轴重合； $U_{120}(010)$ 表示 B 相导通，它的向量与 B 轴重合，所以在空间上与 $U_0(100)$ 相差 120° ，其他向量以此类推；黑色的中间向量，比如 $U_{60}(110)$ 可以看做相邻向量 $U_0(100)$ 和 $U_{120}(010)$ 的合成向量，

$U_Z(111)$ 和 $U_Z(000)$ 是零矢量，他们不产生磁场，对电机不起作用。

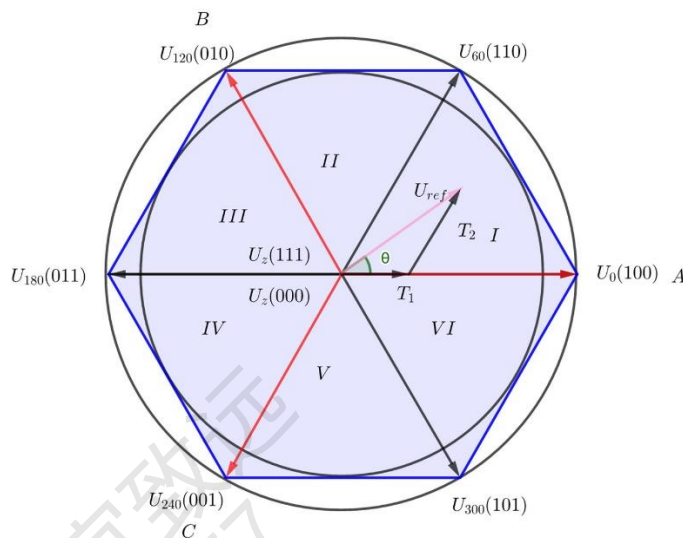


图 3-17 基础矢量大小和位置

根据伏秒平衡原理，在一个开关周期内某个矢量的作用效果等同于两个相邻基础矢量分别作用不同时间的效果之和。

假定某个非零矢量 U_{ref} 位于上面矢量图中的扇区 I 中， T_1 是基础矢量 $U_0(100)$ 的作用时间， T_2 是基础矢量 $U_{60}(110)$ 的作用时间， T 为一个开关周期，那么有以下等式成立：

$$\int_0^T U_{ref} dt = \int_0^{T_1} U_0(100) dt + \int_{T_1}^{T_1+T_2} U_{60}(110) dt + \int_{T_1+T_2}^T U_Z(000) dt$$

也就是参考矢量 U_{ref} 在 T 时间内的作用效果，等同于它的相邻基础矢量 $U_0(100)$ 在 T_1 时间内的作用效果，加上基础矢量 $U_{60}(110)$ 在 T_2 时间内的作用效果之和。

推广到全空间矢量计算， U_Z 表示零矢量， T_Z 表示零矢量的作用时间， U_x 和 U_y 分别是某一个扇区中的两个相邻矢量， T_x 和 T_y 分别

是它们的作用时间， T_s 是合成矢量 U_{ref} 的作用时间，可以推出：

$$U_{ref}T_s = U_xT_x + U_yT_y + U_zT_z$$

从上式可以看出，只要控制相邻矢量的作用时间，也就是控制相应桥臂的导通时间，理论上可以合成平面圆内任意的非零电压矢量。

因此我们可以利用这六个基础矢量与零矢量合成需要的矢量。

3.9 矢量所处扇区的判断

通过以上分析，平面圆内任意非零矢量都可以利用与它相邻的两个基础矢量合成而来，接下来问题是如何确定希望合成的非零矢量所处的扇区。

接着上面的分析，假定位于扇区 I 中的某个非零矢量 U_{ref} ，与它相邻的两个非零矢量为 $U_0(100)$ 和 $U_{60}(110)$ ，它们之间的角度为 60° ：

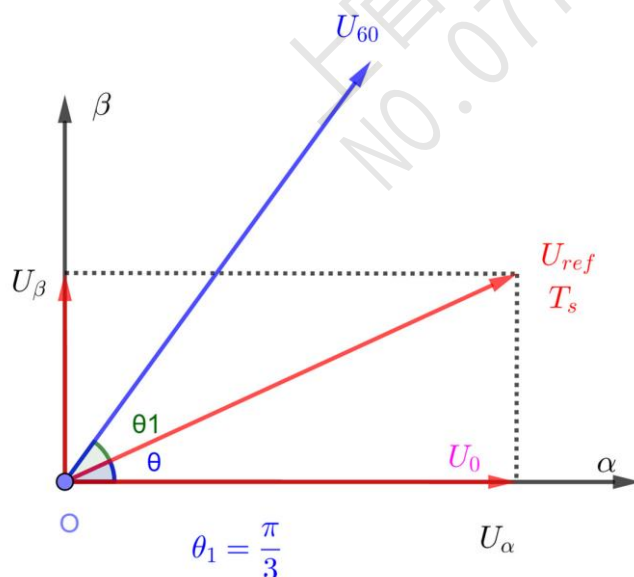


图 3-18 矢量合成分析

因此在扇区 I 中满足 $0 \ll \arctan\left(\frac{U_\beta}{U_\alpha}\right) \ll \frac{\pi}{3}$ 。矢量落在扇区 I 的充分必要条件是： $U_\alpha > 0, U_\beta > 0$ 且 $\frac{U_\beta}{U_\alpha} < \sqrt{3}$ ，同理可以推出矢量落在其它扇区的充分必要条件，如下表所示：

扇区	充要条件
I	$U_\alpha > 0, U_\beta > 0$ 且 $\frac{U_\beta}{U_\alpha} < \sqrt{3}$
II	$U_\beta > 0$ 且 $\frac{U_\beta}{ U_\alpha } > \sqrt{3}$
III	$U_\alpha < 0, U_\beta > 0$ 且 $-\frac{U_\beta}{U_\alpha} < \sqrt{3}$
IV	$U_\alpha < 0, U_\beta < 0$ 且 $\frac{U_\beta}{U_\alpha} < \sqrt{3}$
V	$U_\beta < 0$ 且 $-\frac{U_\beta}{ U_\alpha } > \sqrt{3}$
VI	$U_\alpha > 0, U_\beta < 0$ 且 $-\frac{U_\beta}{U_\alpha} < \sqrt{3}$

表 3-2 扇区与变量充要条件

进一步分析可以看出，合成矢量 U_{ref} 所在的扇区完全由 U_β 、 $\sqrt{3}U_\alpha - U_\beta$ 以及 $-(\sqrt{3}U_\alpha + U_\beta)$ 决定，因此为了继续简化计算，做如下定义：

$$\begin{cases} U_1 = U_\beta \\ U_2 = \frac{\sqrt{3}U_\alpha - U_\beta}{2} \\ U_3 = -\frac{\sqrt{3}U_\alpha + U_\beta}{2} \end{cases}$$

并且满足 $U_1 + U_2 + U_3 = 0$ 。为了继续简化计算，我们规定若 $U_1 > 0$ ，则 $A = 1$ ；否则 $A = 0$ ；若 $U_2 > 0$ ， $B = 1$ ；否则 $B = 0$ ；若 $U_3 > 0$ ，则 $C = 1$ ；否则 $C = 0$ ；因此 A、B、C 共有八种组合，但是由上面的表格扇区计算中不存在 A、B、C 全是 0 或者全是 1 的状态，因此有

效的 A、B、C 组合一共有六种。

令 $N = A + 2B + 4C$ 通过 A、B、C 的不同组合计算出 N 值，可以很方便的判断矢量所处的扇区：

扇区	N值	ABC值	$U_x(x=1,2,3)$ 大小
I	3	$A = 1, B = 1, C = 0$	$U_1 > 0, U_2 > 0, U_3 < 0$
II	1	$A = 1, B = 0, C = 0$	$U_1 > 0, U_2 < 0, U_3 < 0$
III	5	$A = 1, B = 0, C = 1$	$U_1 > 0, U_2 < 0, U_3 > 0$
IV	4	$A = 0, B = 0, C = 1$	$U_1 < 0, U_2 < 0, U_3 > 0$
V	6	$A = 0, B = 1, C = 1$	$U_1 < 0, U_2 > 0, U_3 > 0$
VI	2	$A = 0, B = 1, C = 0$	$U_1 < 0, U_2 > 0, U_3 < 0$

表 3-3 扇区与 N 值的对应关系

采用上述方法，经过简单的加减逻辑运算便可以判断出希望合成矢量所在的扇区。

3.10 矢量作用时间分析

通过以上分析，平面圆内任意非零矢量都可以利用与它相邻的两个基础矢量合成而来，确定了矢量所处的扇区之后，接下来是确定与它相邻的两个基础矢量作用时间的长短。

假定我们需要合成的旋转电压矢量的角速度为 $\omega = 2\pi f$ ， f 为旋转电压的频率，即所形成的正弦电压矢量的频率，它的周期为 $T = \frac{1}{f}$ 。假定载波频率为 f_s ，就是我们的 PWM 的频率，那么频率比 $R = \frac{f_s}{f}$

也就是我们将平面圆平均等分成 R 份，那么每一份的角度递增为：

$$d\theta = \frac{2\pi}{R} = 2\pi \frac{f}{f_s}$$

设 T_s 为载波信号的周期，则 $T_s = \frac{1}{f_s}$ ，那么

$$d\theta = \frac{2\pi}{R} = 2\pi \frac{f}{f_s} = 2\pi \frac{T_s}{T}$$

这样当合成矢量走过整个圆周后，就会在空间形成一个旋转的电压矢量，从而形成一个近似圆形的旋转磁场。

假定欲合成位于扇区 I 中的某个非零矢量 U_{ref} ， T_4 是基础矢量 $U_0(100)$ 的作用时间， T_6 是基础矢量 $U_{60}(110)$ 的作用时间， T 为一个开关周期， T_0 和 T_7 分别为两个零矢量 $U_z(000)$ $U_z(111)$ 的作用时间。

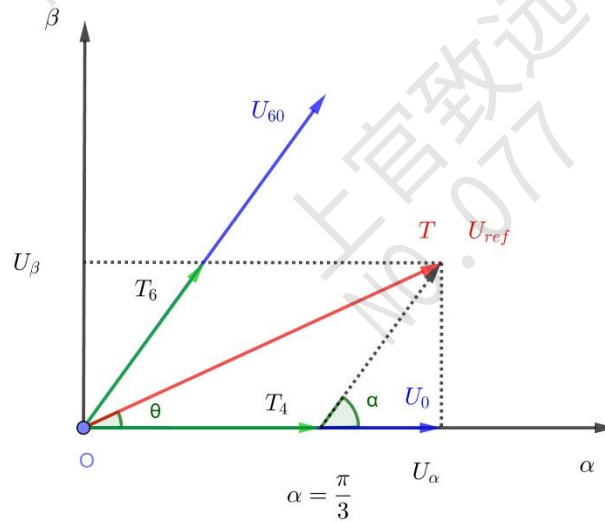


图 3-19 矢量合成时间

非零矢量 U_{ref} 与 α 轴的夹角为 θ ，由正弦定理可以推出：

$$\frac{|U_{ref}|}{\sin \frac{2\pi}{3}} = \frac{\frac{T_4}{T} |U_0(100)|}{\sin(\frac{\pi}{3} - \theta)} = \frac{\frac{T_6}{T} |U_{60}(110)|}{\sin(\theta)}$$

由于基础矢量的模长为 $\frac{2}{3}U_{dc}$ ，所以：

$$|U_0(100)| = |U_{60}(110)| = \frac{2}{3}U_{dc}$$

代入上式可以推出：

$$\begin{cases} T_4 = \sqrt{3} \frac{|U_{ref}|}{U_{dc}} T \sin\left(\frac{\pi}{3} - \theta\right) \\ T_6 = \sqrt{3} \frac{|U_{ref}|}{U_{dc}} T \sin(\theta) \end{cases}$$

两个零矢量的作用时间为： $T_0 = T_7 = \frac{T - T_4 - T_6}{2}$ 。

定义 $m = \sqrt{3} \frac{|U_{ref}|}{U_{dc}}$ 为 SVPWM 的调制系数，由于 $T_4 + T_6 \leq T$ ，因此

$$\sqrt{3} \frac{|U_{ref}|}{U_{dc}} \sin\left(\frac{\pi}{3} - \theta\right) + \sqrt{3} \frac{|U_{ref}|}{U_{dc}} \sin(\theta) \leq 1$$

化简后得出：

$$\sqrt{3} \frac{|U_{ref}|}{U_{dc}} \cos\left(\frac{\pi}{6} - \theta\right) \leq 1$$

$\theta = \frac{\pi}{6}$ 时， $T_4 + T_6 = T$ 输出矢量的最大幅值为 $|U_{ref}| = \frac{\sqrt{3}}{3}U_{dc}$ ，也就是可以输出的最大相电压为 $\frac{\sqrt{3}}{3}U_{dc}$ ，可输出的最大线电压为：

$$\sqrt{3}U_{ref} = \sqrt{3} \frac{U_{dc}}{\sqrt{3}} = U_{dc}$$

相比于传统的 SPWM 线电压最大为 $\frac{\sqrt{3}}{2}U_{dc}$ 来看，SVPWM 的直流母线利用率更高，高出 15.47%：

$$\frac{U_{dc}}{\frac{\sqrt{3}}{2}U_{dc}} = \frac{2}{\sqrt{3}} \approx 1.1547$$

知道了合成矢量的空间角度，那么相邻基础矢量的时间也就能确定下来。

由于常见的 MCU 不具备浮点运算能力，计算三角函数常常使用查表的方式实现。于是人们想到，能不能不使用三角函数，只通过简单的加减逻辑运算来确定基础矢量的作用时间。

进一步分析，仍然以第一扇区为例，将合成矢量分别投影到 α 、 β 轴，那么可以推出：

$$\begin{cases} U_{\alpha}T = U_{ref} \cos \theta T \\ U_{\beta}T = U_{ref} \sin \theta T \end{cases}$$

基础矢量 $U_0(100)$ 的作用时间为 T_4 ，全分量在 α 轴；基础矢量 $U_{60}(110)$ 的作用时间为 T_6 ，在 α 轴的分量为 $U_{60}(110) \cos \frac{\pi}{3} T_6$ ，在 β 轴的分量为 $U_{60}(110) \sin \frac{\pi}{3} T_6$ ，因此上式可以写成：

$$\begin{cases} U_{\alpha}T = U_{ref} \cos \theta T = \frac{2}{3} U_{dc} T_4 + \frac{2}{3} U_{dc} \cos \frac{\pi}{3} T_6 \\ U_{\beta}T = U_{ref} \sin \theta T = \frac{2}{3} U_{dc} \sin \frac{\pi}{3} T_6 \end{cases}$$

写成矩阵形式为：

$$\begin{bmatrix} U_{\alpha} \\ U_{\beta} \end{bmatrix} T = U_{ref} \begin{bmatrix} \cos \theta \\ \sin \theta \end{bmatrix} T = \frac{2}{3} U_{dc} \begin{bmatrix} 1 \\ 0 \end{bmatrix} T_4 + \frac{2}{3} U_{dc} \begin{bmatrix} \cos \frac{\pi}{3} \\ \sin \frac{\pi}{3} \end{bmatrix} T_6$$

进一步整理后得：

$$\begin{cases} U_{\alpha}T = \frac{2}{3} U_{dc} \left(T_4 + \frac{1}{2} T_6 \right) \\ U_{\beta}T = \frac{2}{3} U_{dc} \left(\frac{\sqrt{3}}{2} T_6 \right) \end{cases}$$

因此可以推出两个基础矢量与零矢量的作用时间：

$$\begin{cases} T_4 = \frac{\sqrt{3}}{U_{dc}} \left(\frac{\sqrt{3}U_\alpha - U_\beta}{2} \right) T \\ T_6 = \frac{\sqrt{3}U_\beta}{U_{dc}} T \\ T_0 = T_7 = \frac{T - T_4 - T_6}{2} \end{cases}$$

根据上一节的定义：

$$\begin{cases} U_1 = U_\beta \\ U_2 = \frac{\sqrt{3}U_\alpha - U_\beta}{2} \\ U_3 = -\frac{\sqrt{3}U_\alpha + U_\beta}{2} \end{cases}$$

另外定义 $K = \frac{\sqrt{3}}{U_{dc}} T$ ，因此基础矢量的作用时间又可以写成：

$$\begin{cases} T_4 = \frac{\sqrt{3}}{U_{dc}} U_2 T = U_2 K \\ T_6 = \frac{\sqrt{3}}{U_{dc}} U_1 T = U_1 K \\ T_0 = T_7 = \frac{T - T_4 - T_6}{2} \end{cases}$$

同理可以推出矢量在其他扇区时，相邻基础矢量的作用时间。

第二扇区：

$$\begin{cases} T_6 = -\frac{\sqrt{3}}{U_{dc}} U_3 T = -U_3 K \\ T_2 = -\frac{\sqrt{3}}{U_{dc}} U_2 T = -U_2 K \\ T_0 = T_7 = \frac{T - T_2 - T_6}{2} \end{cases}$$

第三扇区：

$$\begin{cases} T_2 = \frac{\sqrt{3}}{U_{dc}} U_1 T = U_1 K \\ T_3 = \frac{\sqrt{3}}{U_{dc}} U_3 T = U_3 K \\ T_0 = T_7 = \frac{T - T_2 - T_3}{2} \end{cases}$$

第四扇区：

$$\begin{cases} T_1 = -\frac{\sqrt{3}}{U_{dc}} U_1 T = -U_1 K \\ T_3 = -\frac{\sqrt{3}}{U_{dc}} U_2 T = -U_2 K \\ T_0 = T_7 = \frac{T - T_1 - T_3}{2} \end{cases}$$

第五扇区：

$$\begin{cases} T_1 = \frac{\sqrt{3}}{U_{dc}} U_3 T = U_3 K \\ T_5 = \frac{\sqrt{3}}{U_{dc}} U_2 T = U_2 K \\ T_0 = T_7 = \frac{T - T_1 - T_5}{2} \end{cases}$$

第六扇区：

$$\begin{cases} T_4 = -\frac{\sqrt{3}}{U_{dc}} U_3 T = -U_3 K \\ T_5 = -\frac{\sqrt{3}}{U_{dc}} U_1 T = -U_1 K \\ T_0 = T_7 = \frac{T - T_4 - T_5}{2} \end{cases}$$

从以上分析可知，当两个零矢量的作用时间为 0 时，非零矢量的作用时间最长，此时合成矢量的幅值最大，从矢量圆中可以看

出，合成矢量的最大幅值不会超过六个基础矢量所决定的圆形区域。SVPWM 能够输出的最大不失真电压矢量是正六边形的内切圆。

由图中可以推出，内切圆的半径为：

$$\frac{2}{3} U_{dc} \times \sin \frac{\pi}{3} = \frac{\sqrt{3}}{3} U_{dc}$$

这个值就是逆变器输出的最大不失真相电压的幅值，和前面的计算一致。

当合成矢量落在正六边形与正六边形的外切圆形之间的区域时，将会发生过调制，此时必须采取过调制处理，可以采取等比例缩小的方式。设先发生的矢量为 T_x 后发生的矢量为 T_y ；

当 $T_x + T_y > T$ 时，矢量落在六边形与其外切圆之间，发生过调制，波形出现失真，假设不失真时的作用时间分别为 T_x' 、 T_y' ，

那么：

$$\frac{T_x'}{T_x} = \frac{T_y'}{T_y}$$

可以推出缩小后的矢量作用时间：

$$\begin{cases} T_x' = \frac{T_x}{T_x + T_y} \times T \\ T_y' = \frac{T_y}{T_x + T_y} \times T \\ T_0 = T_7 = 0 \end{cases}$$

3.11 七段式 PWM 调制方式分析

确定了合成矢量所需要的相邻基础矢量的作用时间后，接下来的问题是，怎么和 MCU 的 PWM 结合起来。

由上一节的分析可知，某一扇区的两个相邻基础矢量，无论谁先导通，理论上都可以得出对应的合成矢量，但是为了更好的减少谐波，获得更好的效果，一般采取以下原则进行脉宽调制的处理：

- a. 每次矢量变化只改变一个桥臂上的一组 MOS 管；
- b. 合理利用零矢量，尽可能减少开关管的切换次数，减少开关管通断过程中所造成的损耗；
- c. 尽可能减少系统谐波分量，特别是低频信号的谐波分量；
- d. 矢量合成的算法易于实现，减少 CPU 处理时间。

鉴于以上原则，目前有两种方式普遍被大家采用，一种是对称式调制模式也称为七段式调制，一种是低开关损耗模式也称为五段式调制。七段式调制方式开关次数比五段式多一些，但是波形对称，谐波分量小，目前应用较多。

七段式调制，顾名思义，它的 PWM 三相波形呈对称发布，且一个周期由七段组成，共需要六次开关切换，每次开关状态转换，只改变其中一相的开关状态， $U_z(000)$ 零矢量放在两边，且左右对称，中间是零矢量 $U_z(111)$ 可以有效的降低 PWM 的谐波分量。

当 $U_0(100)$ 切换到零矢量 $U_z(000)$ 时，只需要改变 A 相上下桥臂一组开关管的状态切换，若 $U_0(100)$ 切换到 $U_z(111)$ 则需要改变 B 和 C 两相的开关管状态切换，因此需要合理的分配 $U_0(100)$ 、

$U_z(000)$ 和 $U_z(111)$ 的作用时间, 使得每次只切换一组开关管的状态, 这样才能最低限度的减少开关管切换次数, 降低系统谐波分量。比如, 在扇区 I 中, 由两个基础矢量 $U_0(100)$ 和 $U_{60}(110)$ 组成, 根据以上开关切换原则, 配合零矢量, 且每次只改变一组开关, 那么扇区 I 的开关切换顺序为:

$$U_z(000) \rightarrow U_0(100) \rightarrow U_{60}(110) \rightarrow U_z(111) \rightarrow U_{60}(110) \rightarrow U_0(100) \rightarrow U_z(000)$$

且只有这一种切换顺序, 中间的 $U_z(111)$ 零矢量有两个作用时间, 且不需切换, 一共是七段波形, 六次切换。

由此可以推出矢量在扇区 I 时, 七段式 PWM 的配置方式, 其他扇区以此类推。

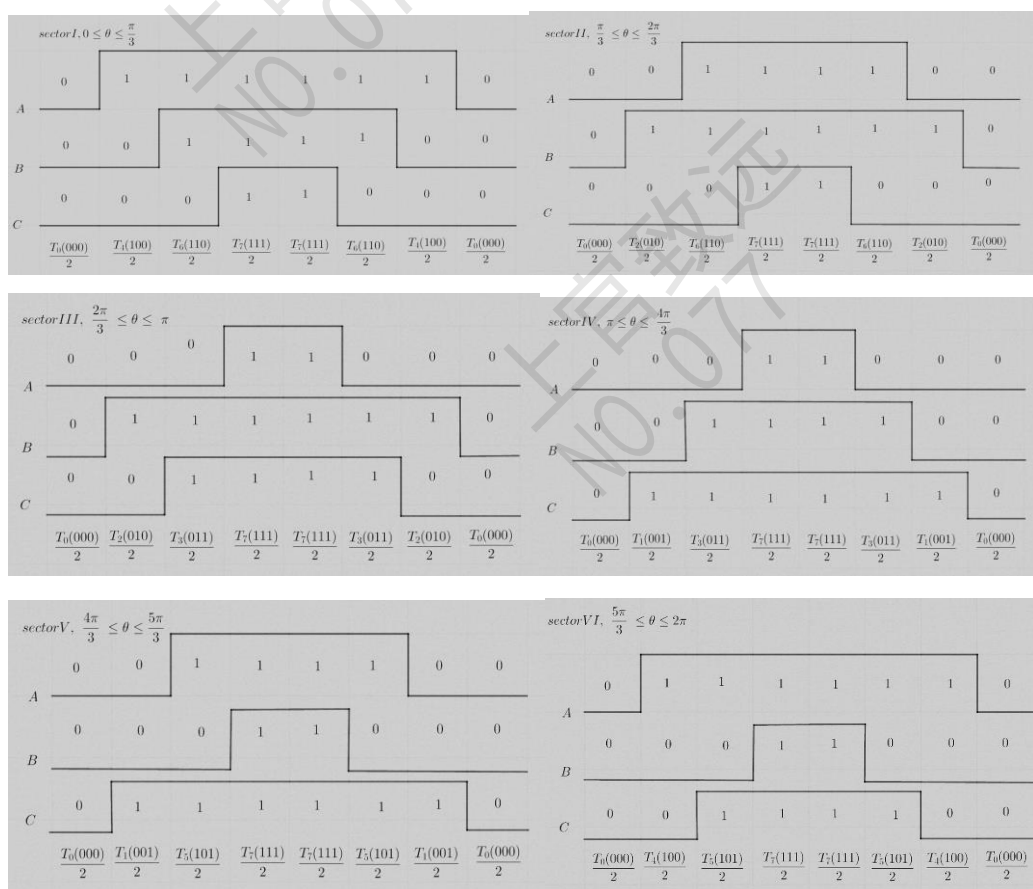


图 3-20 七段式 PWM 波形调制

确定了 PWM 脉宽调制方式之后，接下来我们开始计算一下对应的 PWM 的比较器的值，这是 SVPWM 最后一步也是最关键的一步。

由七段式 PWM 调制方式看出，对称式调试，两个基础矢量和两个零矢量都是对称作用，当采用 PWM 增计数模式时，也就是正三角方式，从扇区 I 中可以推出，A 比较器率先达到给定的值，B 比较器紧随其后，最后是 C 比较器，三路比较器都是对称发波：

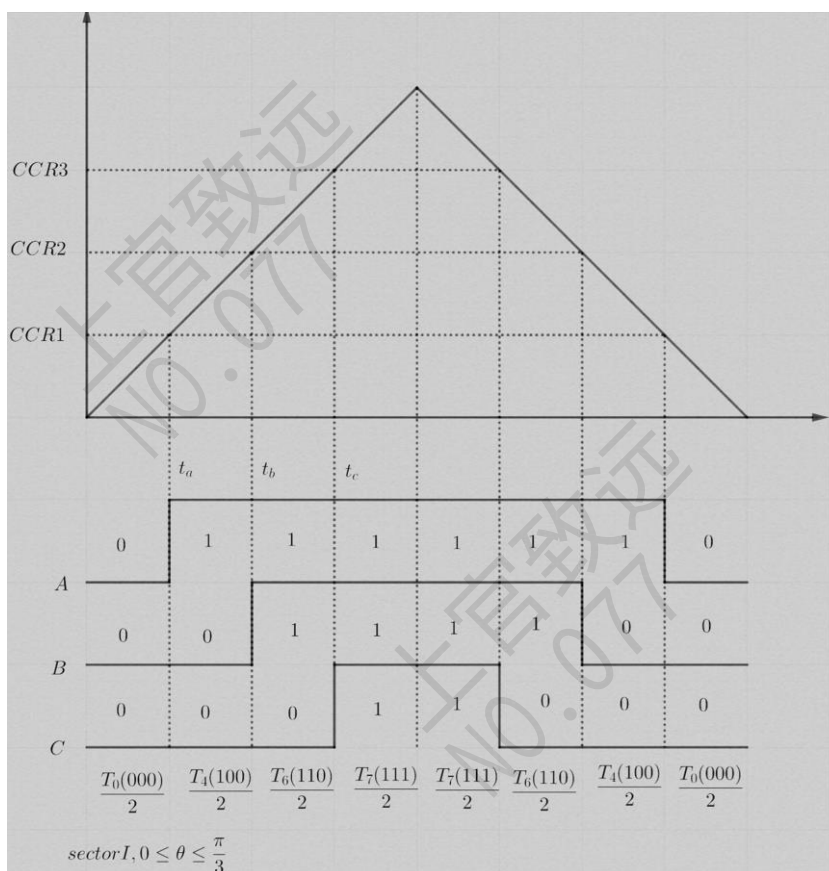


图 3-21 PWM2 模式下扇区 I 对应的占空比分析

由此可以推出：

$$\begin{cases} T_a = \frac{(T - T_x - T_y)}{4} \\ T_b = T_a + \frac{T_x}{2} \\ T_c = T_b + \frac{T_y}{2} \end{cases}$$

t_a 表示先作用电压矢量的开始时刻，因为一个周期由两个零矢量，且左右对称，等分成 4 份，因此上式除以 4； t_b 表示次作用电压矢量的开始时刻； t_c 表示零矢量 $U_z(111)$ 开始作用的时刻。 T_x 、 T_y 分别是先后作用的两个基础矢量的作用时间。 T_a 、 T_b 、 T_c 表示三路占空比该模式下， t_a 、 t_b 、 t_c 同样也代表了各自的 CCRx 值。

这样，就可以推出其他扇区时，对应的比较器的值，以七段式方式调制为例。 CCR_A 、 CCR_B 、 CCR_C 表示对应的三路比较器的值：

扇区	I	II	III	IV	V	VI
CCR_A	T_a	T_b	T_c	T_c	T_b	T_a
CCR_B	T_b	T_a	T_a	T_b	T_c	T_c
CCR_C	T_c	T_c	T_b	T_a	T_a	T_b

表 3-4 扇区与对应占空比的值

第二个扇区 B 先于 A 达到计数器的值，随后是 C 到达计数器的值，因此各路比较器的值分别是 T_b 、 T_a 、 T_c ，其他扇区同样可以根据波形推出。

以上为 PWM2 模式的占空比分析，如果是 PWM1 模式时，即 TIM_CNT 小于 CCR 的值时输出有效电平，大于 CCR 小于 ARR 时输出无效电平，那么此时的占空比图形为倒三角，三路矢量开始作用的时刻计算方式不变，计算占空比时注意一下方向即可。

图 3-22 为 PWM1 倒三角模式下的作用时间图，矢量导通的先后时间未变， t_a 仍为先作用的矢量的开始作用时刻； t_b 仍为后作用矢量的开始作用时刻； t_c 仍为零矢量 $U_z(111)$ 开始作用的时刻。

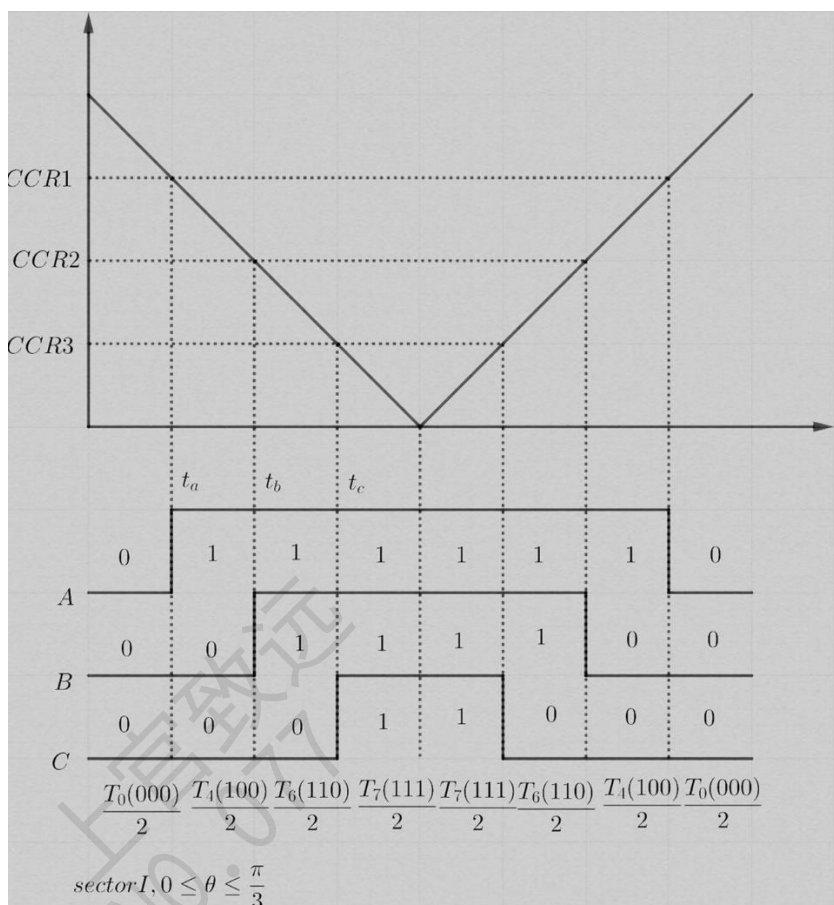


图 3-22 PWM1 模式下扇区 I 的占空比分析

从图中可以看出, CCR1 的值为 $\frac{T_7(111)}{2} + \frac{T_6(110)}{2} + \frac{T_4(100)}{2}$, CCR2 的值为 $\frac{T_7(111)}{2} + \frac{T_6(110)}{2}$, CCR3 的值为 $\frac{T_7(111)}{2}$ 由上面的计算得知, 零矢量的作用时间为 $\frac{T-T_x-T_y}{4}$, 因此可以推出 PWM1 模式下的占空比计算公式:

$$\begin{cases} T_a = \frac{(T + T_x + T_y)}{4} \\ T_b = T_a - \frac{T_x}{2} \\ T_c = T_b - \frac{T_y}{2} \end{cases}$$

同样是七段式 PWM 调制方式, 因此各个扇区对应的占空比顺序和 PWM2 模式一样。

到这里，SVPWM 的原理已经分析完毕，并从七段式 PWM 调制方式分析了对应占空比的计算方法。

3.12 标幺化与 Q15 格式

在无刷电机开发过程中，常常看到标幺化三个字，那么什么是标幺化，为什么要进行标幺化呢？

标幺化是电力系统分析和工程计算中常用的数值表示方法，标幺值是各物理量相对于某个基准值而言，它表示各物理量及参数的相对值。标幺值=有名值/基准值。

举一个简单的例子，实际值为 380KV 的电压，当选取基准值为 110KV 时，那么标幺值为 $\frac{380}{110} = 3.45$ 。

在无刷电机控制系统中，一套电机控制软件可以兼容不同厂家、不同电压、不同功率的电机，使用常见的定点 MCU 处理器处理电压、电流以及其他模型参数，运算起来可能会出现实际值变化范围超过 MCU 字长的问题，难以满足不同电机的变量表示和计算精度等要求。

这就需要标幺化处理。标幺化弱化了不同电机不同变量数值上的差别，实际的计算公式经过标幺化处理，变量的数值形式发生了变化，但公式物理本质没变，所以仍然可以应用各种控制算法，使计算起来简单方便。

一般常见的定点 MCU 处理器中为了计算方便，常用 Q1.15 格式表示一个小于 1 的浮点数。一共 16 位，1 位符号位，其它 15 位为小数位。也就是常见的 S16 格式，范围从 -32768 到 +32768。

比如在 ST 的电机库软件中, 使用 0x49E6 表示 $\frac{1}{\sqrt{3}}$ 即 0.57735。

实际的浮点小数转换成 Q15 的方法, 就是用实际的小数乘以有符号 16 位的最大值: $32768 \times 0.57735 = 0x49E6$ 。

而且电角度也使用 S16 格式表示, $-\pi$ 对应 -32768, π 对应 32768, 这样在定点 MCU 处理器中, 变量都对标 S16 格式, 计算起来大大简化。

特别是 SVPWM 的处理和计算中, 常涉及到 $\sqrt{3}$ 的运算, 此时可以将空间矢量对相电压 $\frac{U_{dc}}{\sqrt{3}}$ 进行标幺化处理, $\frac{\frac{2}{3}U_{dc}}{\frac{U_{dc}}{\sqrt{3}}} = \frac{2}{\sqrt{3}}$, 这样处理可以对消掉三角函数 $\sin \frac{\pi}{3}$ 与 $\cos \frac{\pi}{6}$, 简化运算, 提高处理速度。

一般选取母线电压采集的量程和电流采集的量程作为系统的两个基本物理量, 其他的物理量计算均由这两个推导出来。

3.13 电流环与速度环

电流环作为矢量控制的最内环, 通过采集相电流, 与给定值进行比较, 送入 PI 控制器进行控制, 目的是控制转矩。

它的响应最快, 一般频率在 20K 左右, 电流环的 PI 参数, 虽然可以根据相关公式计算出来, 但是实际开发中, 依然需要动手进行调试。

常用的电流环参数估算方法如下:

$$\begin{cases} K_P = L_s \times \frac{\omega_c}{AB} \times K_P DIV \\ K_I = \frac{R_s \times \omega_c \times K_I DIV}{AB} T \end{cases}$$

其中:

$$AB = \frac{V_{bus}DC \times R_{shunt} \times A_{op}}{3.3}$$

$$A = \frac{V_{bus}DC}{2^{16}}$$

$$B = \frac{R_{shunt} \times A_{op} \times 2^{16}}{3.3}$$

R_s 为相电阻； L_s 为相电感； K_pDIV 为比例系数的分数因子，实际使用时可以灵活配置，比如 $K_p = 0.5$ 时，为了方便运算和提高分辨率，可以令 $K_p = 1024$ ，那么 $K_pDIV = 2048$ 。

T 为电流环的执行时间； ω_c 为电流环的带宽； $V_{bus}DC$ 为直流母线电压； R_{shunt} 为采样电阻， A_{op} 为放大倍数。

此次设计的矢量控制系统为 $i_d = 0$ 控制，因此电流环的控制其实为 q 轴的电流控制。

速度环作为电流环的输入，通过与检测或估算的速度进行比较，将差值送到速度 PI 控制器，输出电流给电流环。速度环的调整频率一般低于电流环，常见的速度环的频率为 2K-10KHZ，可以灵活配置，但要注意速度的检测频率一定要大于速度环的执行频率，否则速度环的调整会无效，造成系统震荡。

第四章 传感器与测速原理

4.1 编码器

在机器人、AGV、数控机床以及高精度运动控制行业中，伺服控制器扮演者重要的角色，它是运动控制的关键组成部分，目前常采用基于矢量控制技术的电流、速度以及位置三闭环控制策略，而速度检测的准确与否直接影响了系统的控制精度。

特别是矢量控制的极低速度处理方法，更是彰显了开发者的技术实力。

直流无刷电机的速度检测通常有模拟测速以及数字测速两种，模拟测速通常使用旋转变压器以及测速发电机实现。由于这两种设备体积较大，安装不方便，还会增加电机转轴的转动惯量，输出信号噪声较大，现实中很少应用。

数字测速主要包括霍尔位置传感器以及旋转编码器两种。霍尔位置传感器分辨率较低，只能给出转子的位置扇区和一些特定的位置角，很难适应高精度伺服控制系统；而旋转编码器能够提供较高的位置分辨率，是目前伺服控制系统中应用较多的一种位置传感器。

旋转编码器常用的有绝对位置型光电编码器和增量型光电编码器。下面以增量型光电编码器为例，分析光电编码器的测速原理。

增量型光电编码器采用光电转换原理，将转速信号转换成脉冲电

信号。如下图所示，增量型光电编码器主要由光源、码盘、光栅、光电检测原件和转换电路组成。当电机轴带动码盘旋转时，光线透过码盘和检测光栅上的缝隙照射到光电检测原件上，输出一组相位差 90° 的类正弦信号，再经过转换电路输出一组规整的正交脉冲信号。增量型编码器光电具有低惯量、低噪声、高精度的优点，但是增量型光电编码器无法输出旋转轴的绝对位置信息。

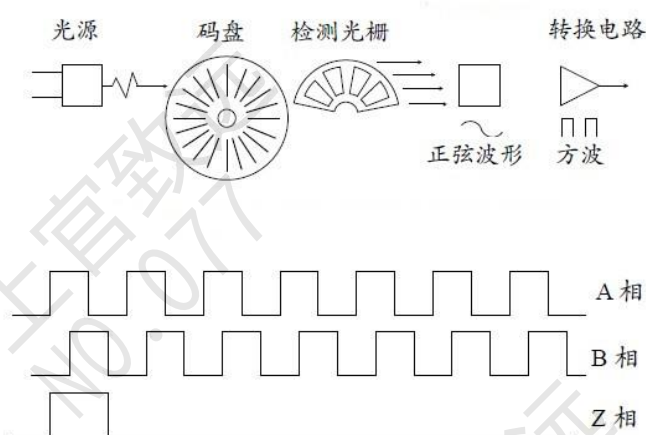


图 4-1 光电编码器

常用的测速原理包括 M 法、T 法、M/T 法等。

4.2 M 法测速

M 法在一定的时间周期 T_c 内，测量编码器输出的脉冲个数 M_1 来计算转速。用个数除以时间就可以得到编码器输出脉冲的频率，因此 M 法也称为频率法， $f_1 = M_1/T_c$ 。

假设电机转动一圈可以产生 Z 个脉冲， $Z = 4 \times$ 编码器线数，这里的 4 表示 4 倍频，因为一般同时采集 A 相和 B 相的上升沿和下降沿。

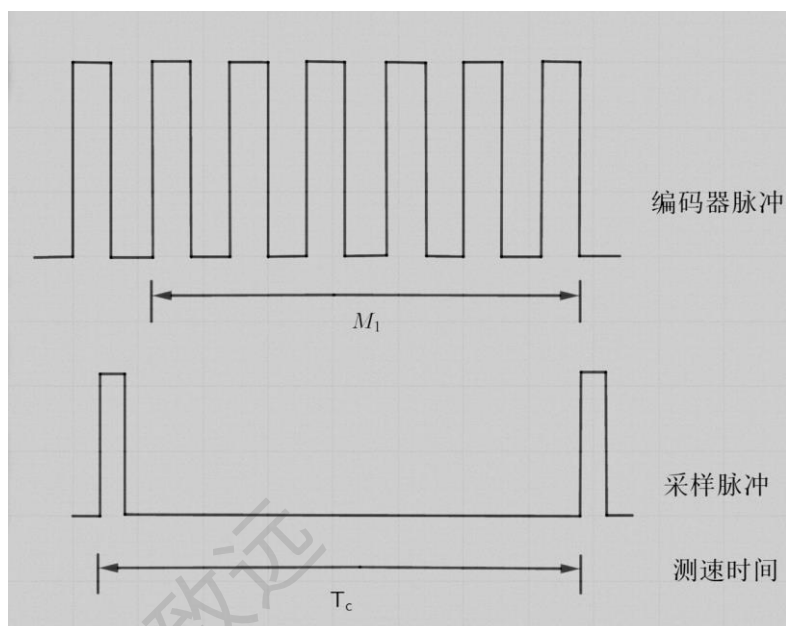


图 4-2 M 法测速原理

用频率 f_1 除以一圈的脉冲个数 Z 就可以得到单位时间内电机的转速：

$$n = \frac{f_1}{Z} \times 60 = \frac{60M_1}{ZT_c}$$

转速的单位为 r/min ，因此后面乘上了一个 60，表示 60 秒，也就是一分钟。一般 Z 和 T_c 为常数，转速与计数脉冲个数 M 成正比，因此这种测速方法被形象的称为 M 法。

当计数值由 M_1 变为 $M_1 + 1$ 时，转速由 $60M_1/ZT_c$ 变为 $60(M_1 + 1)/ZT_c$ ，因此 M 法的分辨率可以这样计算：

$$Q = \frac{60(M_1 + 1)}{ZT_c} - \frac{60M_1}{ZT_c} = \frac{60}{ZT_c}$$

可以看出，M 法测速的分辨率与速度无关，要提高分辨率，即减小 Q 值，必须增大编码器每圈输出的脉冲个数 Z ，或者增大采样周期 T 。

实际上，两个值受到现实条件的制约，不可能无限制的增大。

M 法测速最大误差为一个脉冲，因此误差率的最大值可以这样计算：

$$\delta_{max} = \frac{\frac{60M_1}{ZT_c} - \frac{60(M_1 - 1)}{ZT_c}}{\frac{60M_1}{ZT_c}} \times 100\% = \frac{1}{M_1} \times 100\%$$

测速误差率与脉冲个数成反比关系，转速越高 M 值越大，误差越小；当转速很低时，M 值很小，误差会变大，因此 M 法不适合低速测量，它适合高速测量。

4.3 T 法测速

T 法是测量编码器两个脉冲之间的时间间隔来计算转速，也被称为周期法。实际使用中通过一个高频时钟脉冲的个数 M_2 来计算编码器两个脉冲之间的时间间隔。

假设高频脉冲的频率为 f_0 ，那么两个脉冲之间的时间间隔 $T_t = M_2/f_0$ ，电机的转速可以表示为：

$$n = \frac{60}{ZT_t} = \frac{60f_0}{ZM_2}$$

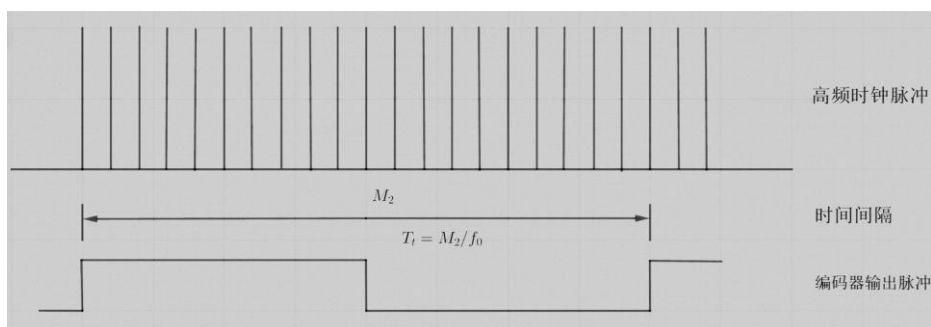


图 4-3 T 法测速原理

当转速变化时, 假设高频时钟脉冲个数由 M_2 变为 $M_2 - 1$ 时, T 法测速的分辨率可以计算为:

$$Q = \frac{60f_0}{Z(M_2 - 1)} - \frac{60f_0}{ZM_2} = \frac{60f_0}{ZM_2(M_2 - 1)}$$

将转速 n 的表达式代入上式可以推出:

$$Q = \frac{Zn^2}{60f_0 - Zn}$$

可以看出, T 法测速的分辨率 Q 与转速有关, 转速越低, Q 值越小, 分辨能力越强。

T 法测速最多可能产生一个脉冲的误差, 因此 T 法测速的最大误差率可以这样计算:

$$\delta_{max} = \frac{\frac{60f_0}{Z(M_2 - 1)} - \frac{60f_0}{ZM_2}}{\frac{60f_0}{ZM_2}} \times 100\% = \frac{1}{M_2 - 1} \times 100\%$$

在低速时, 编码器两个脉冲之间的时间间隔变长, 高频时钟脉冲个数增多, 误差率变小, 因此 T 法测速更适合低速段。

4.4 M/T 法测速

为了兼顾高速与低速, 实际使用中常常是 M 法和 T 法结合使用, 称为 M/T 法测速。

在规定的采样周期 T_c 内, 同时计算编码器脉冲个数 M_1 与高频时钟脉冲个数 M_2 , 两个计数器保持严格同步, 检测时间与编码器输出脉冲保持一致, 最大限度的减小误差。

检测周期由 T_c 采样脉冲开始之后的第一个编码器脉冲上升沿决

定, $T = T_c - \Delta T_1 + \Delta T_2$

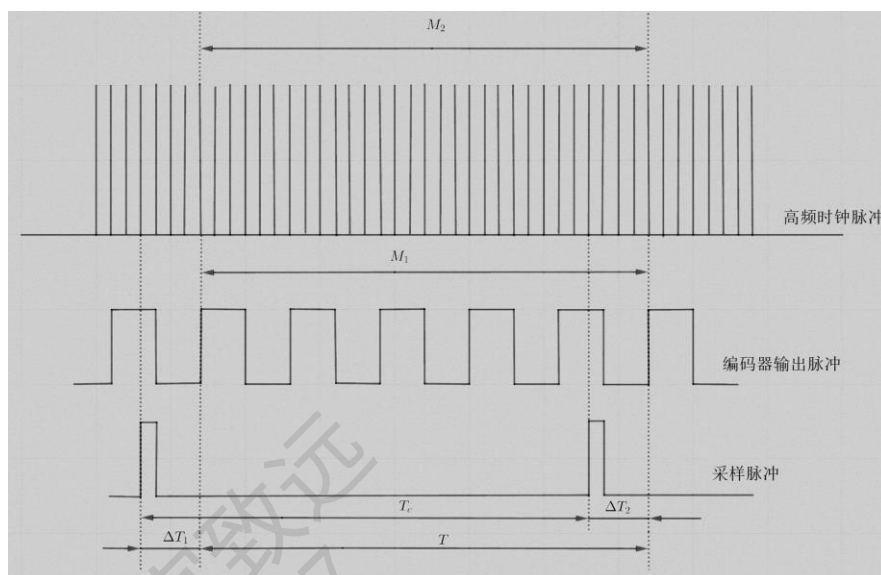


图 4-4 M/T 法测速原理

ΔT_1 是采样脉冲开始时刻到第一个编码器脉冲上升沿之间的时间间隔, ΔT_2 是采样脉冲结束时刻到最后一个编码器脉冲结束时刻之间的时间间隔。

检测周期 T 内电机转过的机械角度为:

$$\theta = \frac{2\pi n T}{60}$$

编码器一转发出 Z 个脉冲, 周期 T 内, 编码器输出的脉冲个数为 M_1 , 那么电机转过的机械角度又可以表示为:

$$\theta = \frac{2\pi M_1}{Z}$$

定义高频时钟脉冲频率为 f_0 , 检测周期 T 内时钟脉冲计数值为 M_2 , 则 $T = M_2/f_0$ 。

综合以上各式可以计算出：

$$n = \frac{60f_0M_1}{ZM_2}$$

高速段， $T_c \gg \Delta T_1$ ，并且 $T_c \gg \Delta T_2$ ，可以认为 $T \approx T_c$ 。

当 M_1 变化一个脉冲时，可以近似认为 M_2 不变，此时的分辨率可以这样计算：

$$Q = \frac{60f_0(M_1 + 1)}{ZM_2} - \frac{60f_0M_1}{ZM_2} = \frac{60f_0}{ZM_2}$$

$M_2 = f_0T \approx f_0T_c$ ，因此上式简化后为：

$$Q = \frac{60}{ZT_c}$$

结果正好与前面 M 法测速分辨率吻合。

当转速很低时， $M_1 = 1$ ，转速的计算公式又和前面 T 法测速等同。因此 M/T 法在低速和高速段都有很高的分辨率，但是速度很低时，要兼顾系统实时性的问题。

随着现代控制技术的进步，涌现出了很多先进的速度检测算法，并在实际工程中得到了很好的应用，比如卡尔曼滤波算法、非线性观测器算法、锁相环算法、模糊控制算法、滑膜观测器算法等等。

4.5 霍尔区间位置估算

在低成本的无刷电机驱动方案中，常常会用到开关型霍尔传感器，用于转子位置检测。由于霍尔传感器分辨率低，难以满足矢量控制高分辨率位置信号的需求，因此需要一种算法，进行霍尔区间位置的估算。

常见得开关型霍尔传感器呈 120° 电气角度摆放，电机的三相分别对应三个霍尔传感器，每一个霍尔传感器一般超前该相 90° 放置。当磁场的南北磁极通过时，霍尔会呈现出高低电平的变化。因此每个电周期，每个霍尔有两个状态输出。

正如前面霍尔部分的分析，一个完整的电气周期，分六个扇区，每个扇区对应不同的三路霍尔组合值，通过检测霍尔的组合值，就可以判断转子当前所处的扇区。

但是它的分辨率只有 $\pm 30^\circ$ ，肯定满足不了矢量控制的要求。通常的做法是通过插值的方法进行转子位置估算，也就是常说的速度对时间的积分。

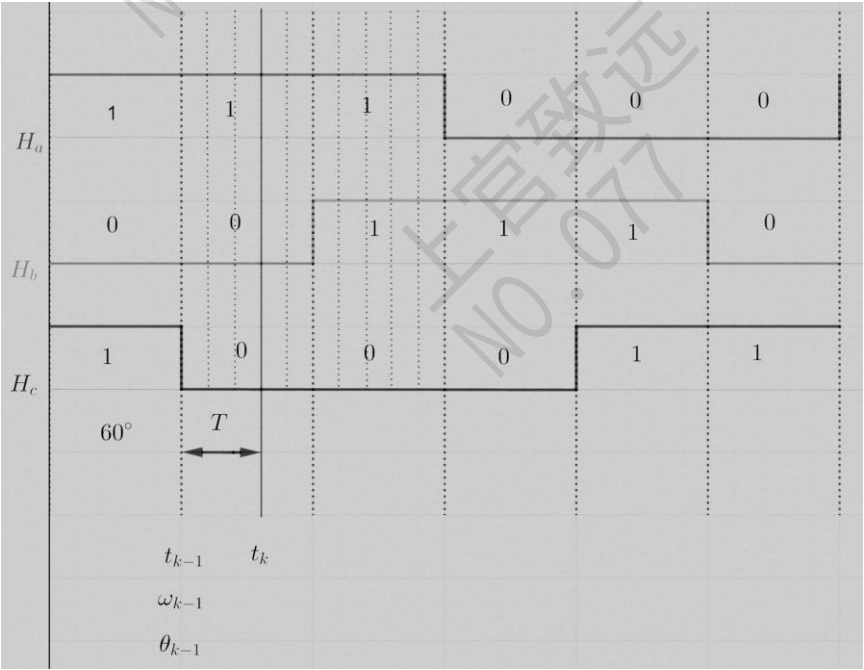


图 4-5 霍尔区间位置估算算法

ω_{k-1} 是 $k-1$ 时刻的转子角速度，实际计算时，这个速度可以通过前一时刻霍尔转过的 60° 扇区所用的时间来计算得到； θ_{k-1} 为 $k-$

1 时刻转子位置，那么在 t_k 时刻，转子的位置可以用下式计算：

$$\theta_k = \theta_{k-1} + \int_{t_{k-1}}^{t_k} \omega_{k-1} dt$$

在程序设计时，插值的细分时间往往是一个矢量控制周期，在每个矢量控制执行周期中进行一次位置的估算，并且在每个 60° 扇区位置进行一次校正。

该算法使用的是霍尔转过 60° 扇区的平均速度，并不是瞬时速度。如果精度要求高，可以考虑加减速的情况，将加速度考虑进去。不过对于一般场合，该算法完全可以拟合出一条很平滑的电角度曲线。

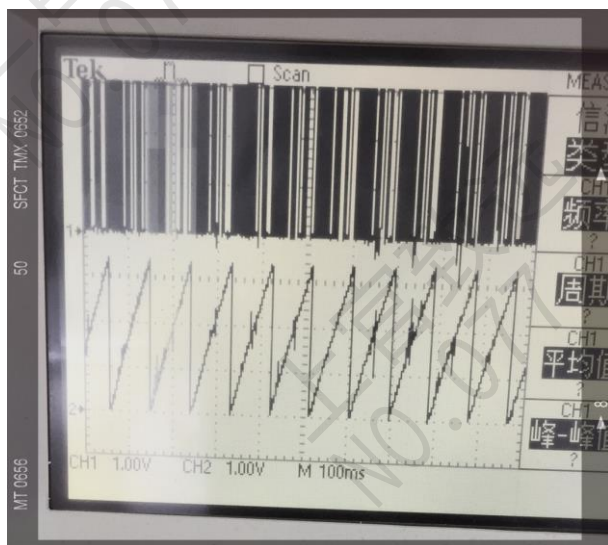


图 4-6 霍尔区间位置估算效果

第五章 电机关键参数的测量

电机参数的测量是正确调试电机的前提，尤其在无感调试中，参数的正确与否直接影响电机能否正常运行。

电机的一些关键参数包括相电阻、相电感、反电动势常数、极对数以及转动惯量等。

测量参数时可以使用 ST 提供的 motor profiler 也可以手动测量。

5.1 相电阻

相电阻的测量可以使用万用表做粗略的测量，或者使用电桥测量。万用表两端分别连接电机的任意两相，所得到的阻值的一半就是相电阻。

5.2 相电感

相电感常使用电桥测量，将频率设为 1KHZ 或以上，可以设置为系统 PWM 所使用的频率，然后测量电机在该频率下的电感。

同样用电桥的两端连接电机的任意两相，得到的电感值的一半就是相电感；可以旋转一圈，多次测量求平均值。

5.3 反电动势常数

一般采用电机对拖的方式测量反电动势常数，小型电机可以使用电动工具带动电机旋转，同时示波器观测电机的反电动势波形，计算波峰与波谷的电压值，并计算此时的转速，反电动势常数由下式计算得出：

$$k_e = \frac{U}{\omega} = \frac{U}{\frac{2\pi}{60} \times n}$$

U 是反电动势的幅值，单位是伏特 V ， ω 是旋转角速度，单位是 rad/s ， n 为转速，单位是 rpm ， k_e 的单位是 $\text{V} \cdot \text{s/rad}$ ，

实际使用中常常使用 rpm 表示电机的速度，使用 V/krpm 表示反电动势常数的单位，它们之间的转换关系如下：

$$1(\text{V/krpm}) = \frac{1}{\frac{2\pi}{60} \times 1000} (\text{Vs/rad})$$

5.4 极对数

电机厂商一般会给出电机的极对数，如果不确定，也可以通过测试得到。可以使用低压直流电源，限制一定的电流，加载到三相中的任意两相，用手转动电机一圈，有稳定位置的个数就是极对数。

当然也可以通过示波器测试，将示波器的端子和地连接到三相电机的任意两相上，用手转动电机一圈，出现几个正弦波，就是几对极。

假如极对数为 p ，那么机械转速与电气速度的关系为：

$$\text{机械转速} \times p = \text{电气速度}$$

5.5 转动惯量

在电机的加减速控制中，常常会涉及到转动惯量的测量。转动惯量是一种表征刚体转动惯性大小的物理量，与刚体的质量、质量相对于转轴的分布等有关。

转动惯量对整个无刷电机控制系统的精度、稳定性以及动态响应都有很大的影响，特别是频繁的正反转切换、频繁的加减速过程中，转动惯量的影响非常明显。

小惯量的电机启动很快，制动性能也好，频繁加减速响应快，适合轻负载、高速度的场合。但是如果负载很大，电机惯量很小，那么这时候可能会对电机轴造成损坏，因此电机的转动惯量与折算到电机轴的负载惯量应该合理匹配，过大过小都不合适。

常见的无刷电机转动惯量的测定方法包括自减速法、辅助摆锤法以及扭摆法。这里以常用的自减速法为例，简单说明无刷直流电机转动惯量的测试方法。

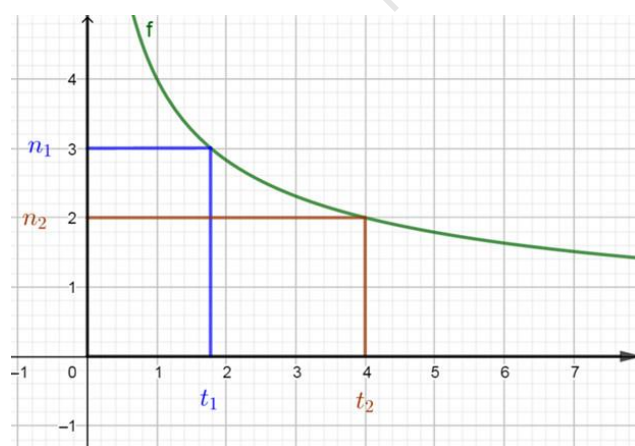


图 5-1 电机减速曲线

给电机额定励磁电流，转速高于额定转速时，让电机空载运转；等电机平稳后迅速切断电机电源，电机在惯性状态下会继续运行，并自然减速，测定电机的减速曲线，如上图所示，电机的转动惯量可以由下面的公式算出：

$$J = \frac{P}{4 \times 1.37 \times \frac{n_1^2 - n_2^2}{t_2 - t_1}} \times 10^6$$

J 为转动惯量，单位是 $\text{kg} \cdot \text{m}^2$ ；P 是额定转速下的机械损耗和空载铁损之和，单位是 kw ； n_1 是 t_1 时刻的转速； n_2 是 t_2 时刻的转速，单位都是 rpm。

第六章 硬件电路设计与实现

6.1 电源电路设计

无刷电机的运行工况复杂，现场常有不同形式的干扰存在，供电电压不稳、浪涌冲击大等常有存在。为了保证无刷电机的正常运行，电源的设计显得尤为重要。

TPS54360 是 TI 公司的新型 BUCK 电源集成模块，具备超宽的电压输入范围，从 4.5V 到 60V，具有可调欠压闭锁功能，因此具备很强的抗浪涌冲击能力。

工作电流小，负载不工作时的电源电流仅为 $146\mu\text{A}$ 左右，效率高达 96%，并具备较宽的输出范围。工业级电源芯片，运行温度从 -40° 到 150° ，具备 3.5A 的持续电流输出。

通过外部不同的配置电阻，可以得到不同的电压输出。下图为典型的 24V 输入，15V 输出电路设计：

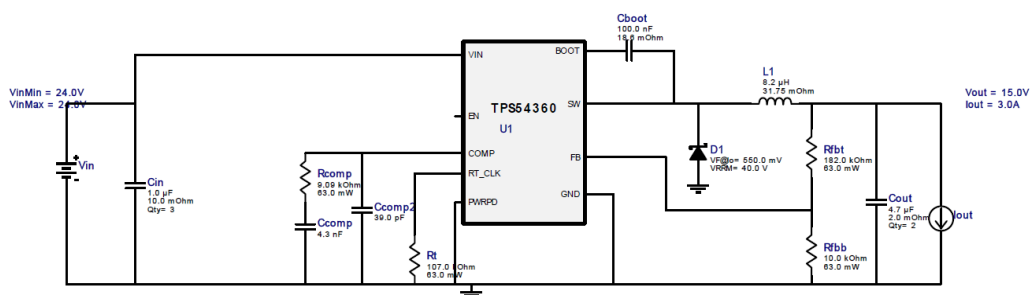


图 6-1 TPS54360 电源电路设计

15V 给逆变器芯片提供电源，系统所需的其它几个电源包括供给 MCU 处理器的 3.3V 电源、另外还有霍尔与编码器所需的 5V 电源等。

5V 电源的产生可以使用常见的 78 系列，本次设计选用 MP2359，一款内置功率 MOSFET 的效率高达 92% 的异步降压型电源转换芯片。具备 4.5V 到 24V 宽范围输入电压，0.8V 到 15V 可调输出，封装小，可输出高达 1.2A 的峰值电流，具备很好的负载和线性调节性能，具备快速瞬态响应的电流控制模式，环路稳定可靠。并带有限流和过温保护。

典型的 5V 输出电路如下图：

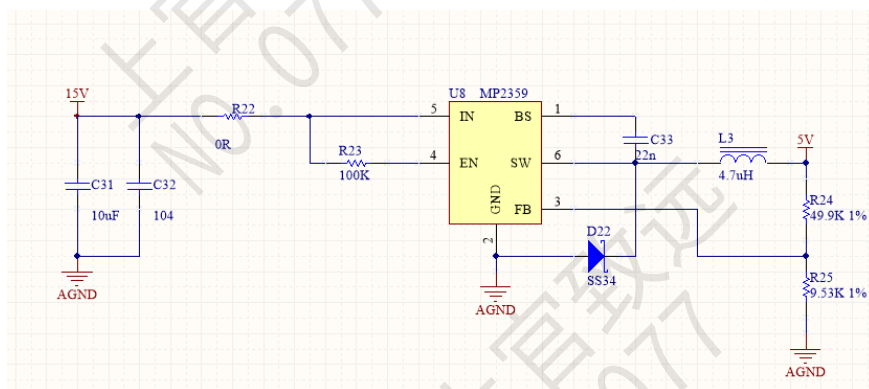


图 6-2 5V 输出电路

特别注意的是，在输入 IN 与使能 EN 之间需要加一个 100K 的电阻，避免芯片使用中出现烧坏现象。

3.3V 的电源用于给 MCU 供电，使用常见的 AMS1117-3.3V 即可。

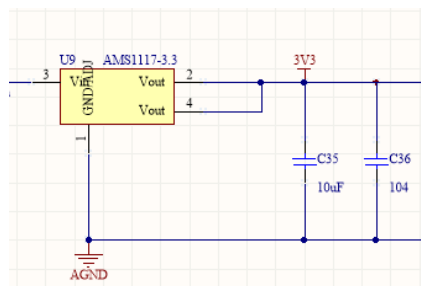


图 6-3 3.3V 输出

6.2 电流采集及其放大电路设计

FOC 的实现需要实时检测相电流，常见的测量方法是在每一相的桥臂下串联一个大功率、低阻值、精密电阻。也有一些低成本领域只使用了单个电阻来检测，也有一些场合使用 ICS 芯片检测电流。本次设计采用常见的三电阻方式实现电流的采集。每次只采集其中两相，另外一相根据基尔霍夫电流定律计算得出。

电阻选用 10 毫欧 2W 1% 精密电阻，功率的选择根据自己设计的最大电流选取。电阻上允许通过的最大电流要大于电机的最大电流，否则常常会出现烧坏电阻的现象。

因为是三电阻采样，所以电流相对于地电平是上下摆动的，因此送到 ADC 模块之前，需要相移电路。所以在运放的同向输入端加入了 1.65V 的电平，这样采集的电流会在 1.65V 上下摆动。

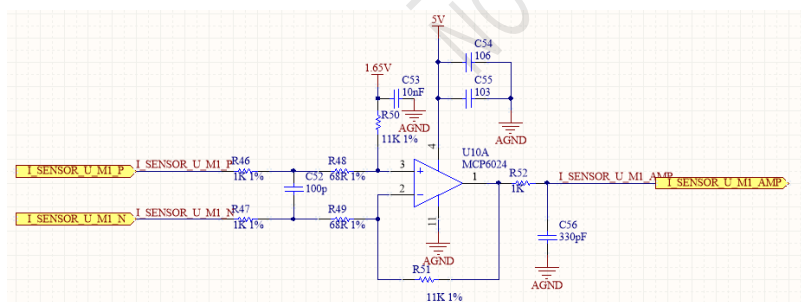


图 6-4 电流采集放大电路

关于放大倍数，并没有一个硬性的要求，本次设计采用的是 10.3 倍， $\frac{R51}{R49+R47} = \frac{11000}{1000+68} = 10.3$ ，放大后以不超过 ADC 的最大输入为标准，如果不太确定怎么计算，可以使用 ST 提供的 motor control

workbench 进行计算：

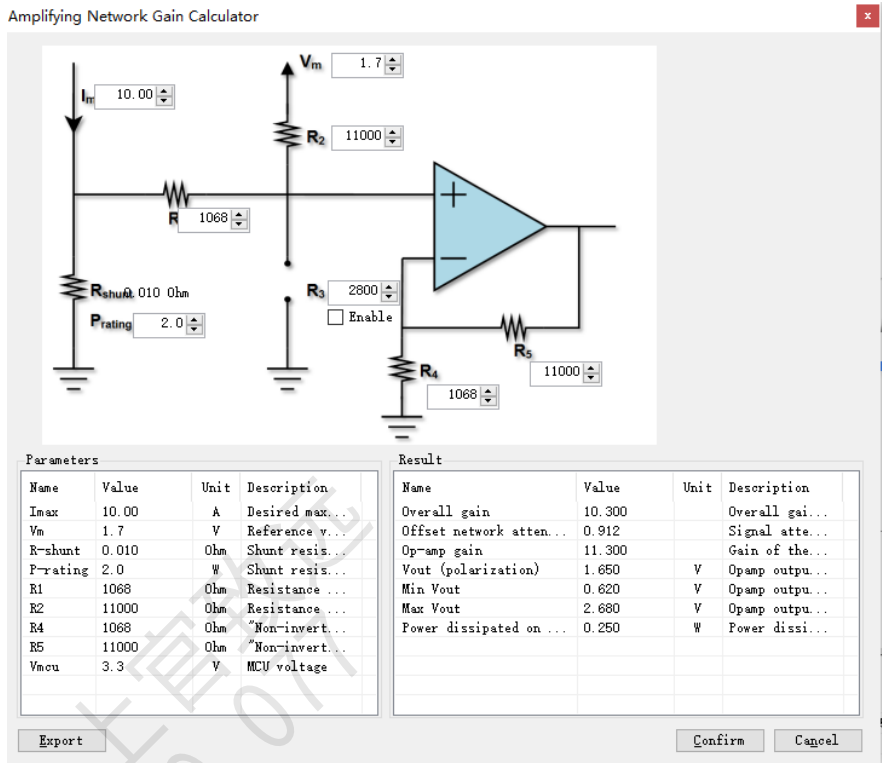


图 6-5 放大倍数与电阻设置

6.3 逆变器驱动电路设计

逆变器的驱动电路常常采用六个功率 MOS 管配合驱动芯片实现。常见的驱动芯片有单路输入互补输出的，也有的芯片自带死区，也有的集成全桥在一个芯片内，总之这部分的实现方式很灵活。

这里以常见的 IR2110S 为例，单路互补输入，互补输出；当然也可以使用集成度更高的 IR2136，一个芯片三路互补输入和输出，大大简化了电路布局。

IR2110S 兼容 3.3V 逻辑电平，门极驱动电源支持从 10v-20v，集成度高，响应速度快，驱动能力强。采用先进的自举电路和电平转换技

术，特别适合 MOS 功率管的驱动。典型的驱动电路如图所示：

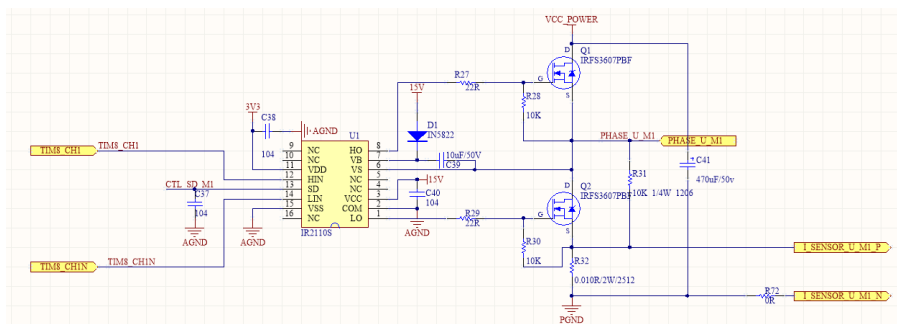


图 6-6 逆变器驱动电路

6.4 过流、过压保护电路设计

过流、过压保护电路可以在系统电流或电压超过预定值时，切断电机控制端，避免过流过压造成系统损坏。常见的过流电路可以这样设计，计算好最大电流，根据采样电阻，换算成电压值，这个值作为阈值给比较器的同相输入端，比较器的另一端接母线采样电阻的一端：

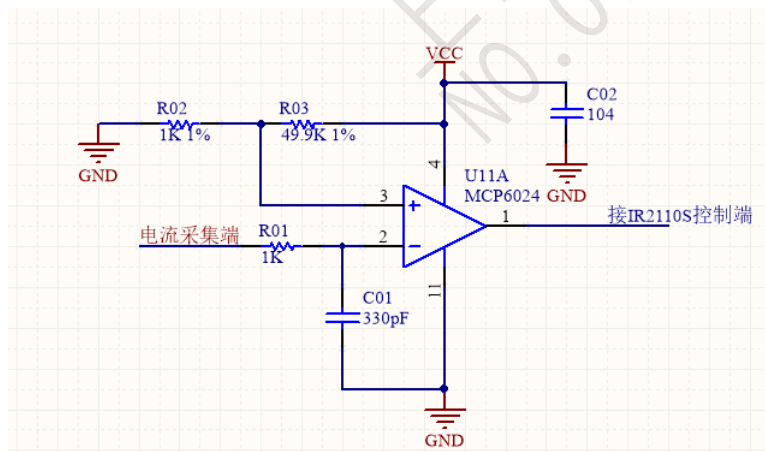


图 6-7 典型的过流保护电路

过压的保护电路设计思路和过流类似，也是通过比较器实现。这里不再赘述。

6.5 MCU 处理器电路设计

处理器电路主要是各模块管脚的正确配置，根据芯片手册，选择对应的 IO 口，另外注意模拟地和数字地的处理，一般用 0 欧姆电阻隔离。各电源端口放置滤波电容，晶振靠近芯片放置。

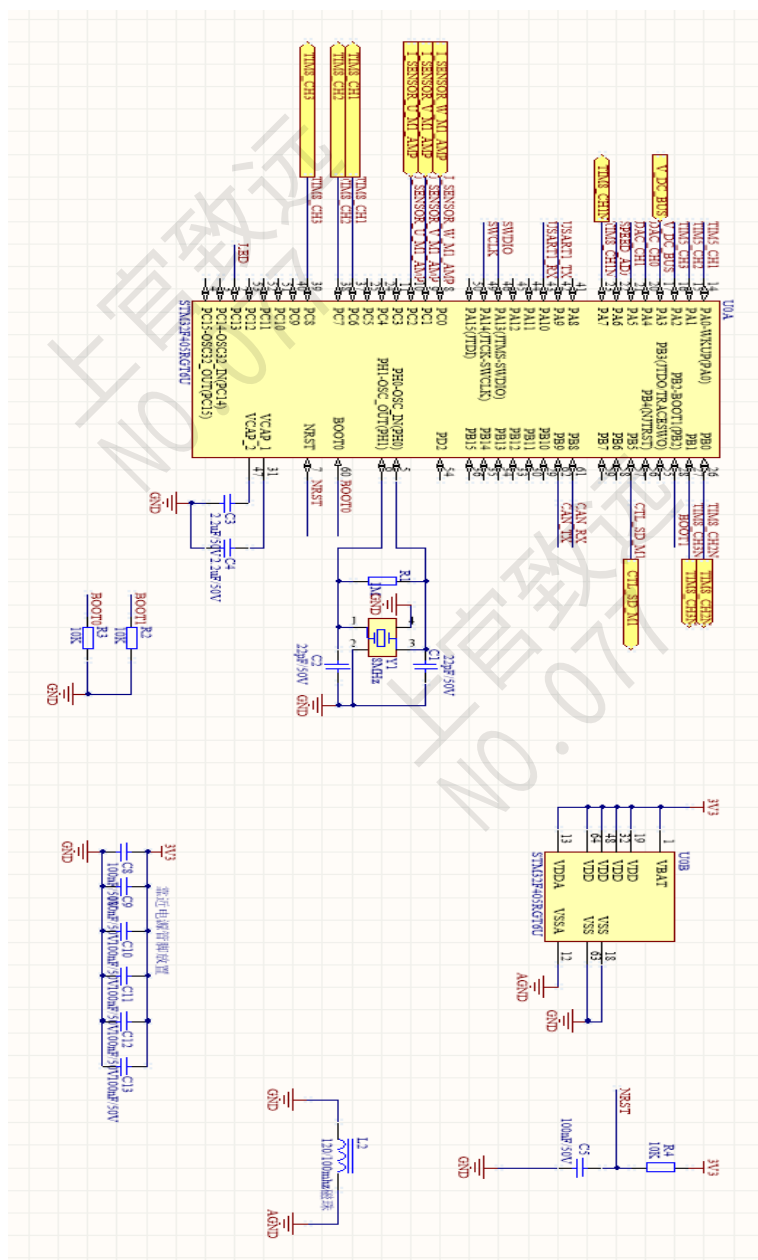


图 6-8 MCU 处理器电路

6.6 通讯接口设计

常见的无刷直流电机驱动的通讯端口为 CAN，通过 CAN 配置电机的一些参数，控制电机的启停和转速等。CAN 的设计可以选用常见的 TJA1050T 芯片实现。

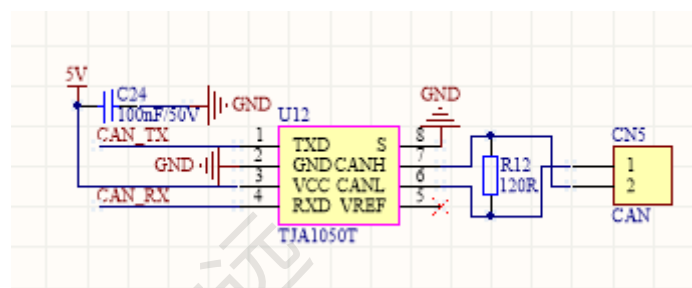


图 6-9 CAN 芯片

为了调试方便，也可以添加 USART 接口，可以添加 232 转换芯片在电路板上，当然为了缩小板子体积，可以只预留 UASRT 的接口。

6.7 PCB 设计注意事项

PCB 设计时，注意电源地、功率地、数字地以及模拟地的隔离。因为功率地会引入电机的一些干扰信号，一般使用一个磁珠和电源地隔离；数字地和模拟地可以采用 0 欧电阻，也可以采用磁珠隔离。

为了增大铜皮承载的电流，设计电机的三相线以及功率线时，常常采用裸露铜皮的做法，并在上面再焊一层锡。裸露的铜皮走线宽度尽量不要超过 2mm，避免干扰的引入。

采样电阻尽量差分走线，走线尽量短，可在一定程度上提高抗干扰能力。采样电阻尽量靠近运算放大器，减少误差和干扰。运放的地与单片机的地单独拉出来接到电源的地。

第七章 程序设计与调试流程

本章以 STM32F405 这款 168MHZ 浮点运算处理器为例，详细分析如何实现 FOC 的各个功能模块。程序的整体架构可以看成两个大的任务处理模块，第一个是 ADC 的中断处理模块，它的频率最高，一般为 PWM 的频率，在 ADC 中断中进行 FOC 的运算。

另外一个任务是状态机的处理，这个任务放在系统滴答定时器中断中实现。

程序中主要参数可通过 DAC 进行调试和观察，比如电流的采样、电角度的计算等等。常见的是使用 DAC 并配合 JLINK 中的 JSCOPE 实现参数的观察和调试，必要时加装简单的电流互感器。当然在调试过程中万用表、示波器都是必不可少的工具。

7.1 程序流程与架构

首先对程序的整体流程有个初步的认识。系统上电之后，需要对时钟以及使用到的各个模块进行初始化配置，ADC、PWM、霍尔传感器接口、通讯接口等。

当模块初始化完毕之后，开始对系统一些参数及变量的初始化配置。其中包括电机初始状态、电流环 PI 参数、速度环 PI 参数等。紧接着进行电流初始偏移量的校正。因为是三电阻采样，电流在零电平上下摆动，因此系统电路里加入了相移电路，整体电平抬高了 1.65V，

因此需要在开始的时候校正该值。

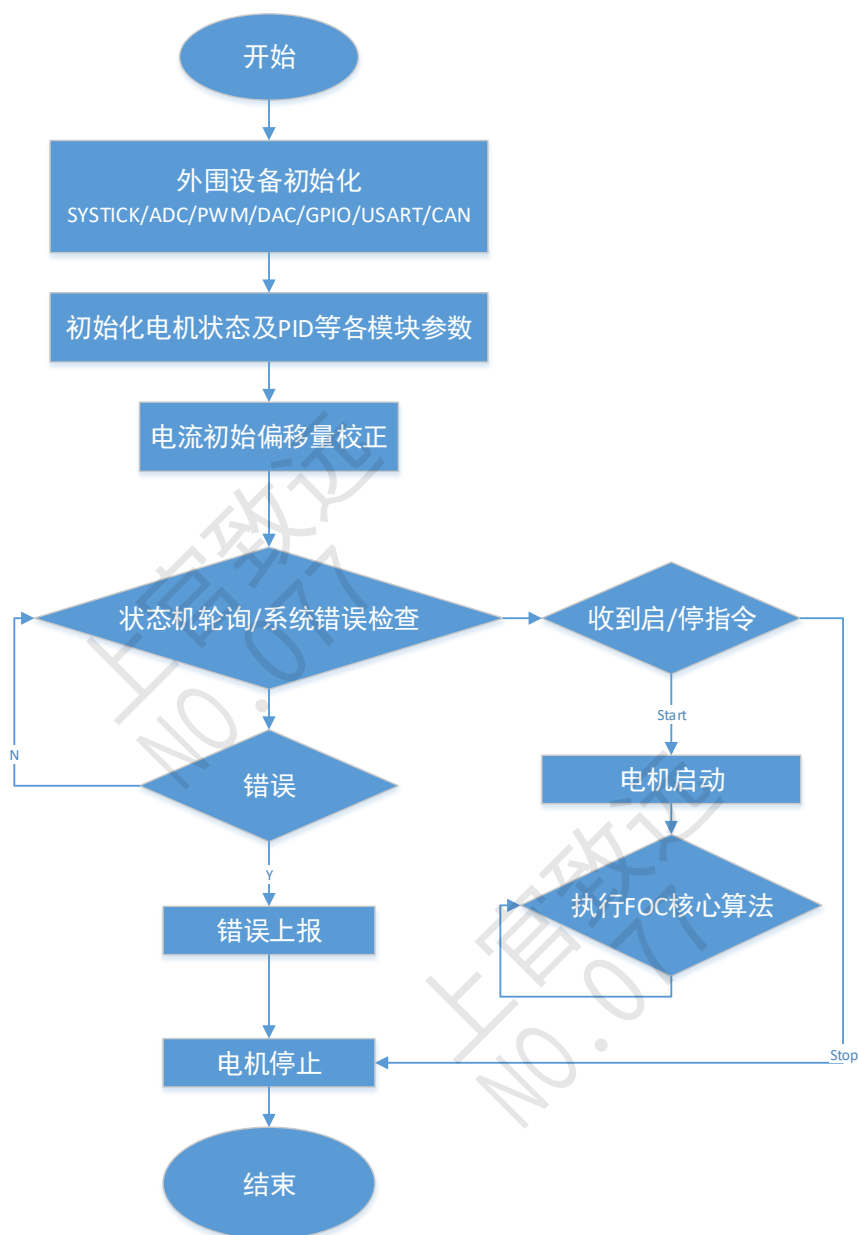


图 7-1 程序流程与架构

系统的状态机轮询任务建议放在 SYSTICK 中断中，当然也可以放在主程序的 while 循环中。状态机轮询主要是电机的各个状态，空闲、启动、停止、错误等。实时检测系统的错误，包括过流、过压、欠压

等。当接收到电机启动命令后,发送启动信号,触发电机运行的任务,开始执行 FOC 核心算法。当接收到停止的命令后,即刻切断 PWM 信号,停止电机的运行,电机运行任务挂起。

如果系统有错误,立即切断 PWM 信号,停止电机,在人为修复该错误后,系统恢复正常状态。

7.2 系统时钟及总线模块配置

STM32F4 系列的时钟系统有点复杂,各个外围模块分别配置使用不同的时钟,因为并不是所有的外设都需要系统时钟这么高的频率,当时钟越快时,系统功耗就会越大,相应的抗电磁干扰能力就会越弱,因此很多复杂的处理器都采用多时钟源的配置方案。

STM32F4 的外设时钟大多是通过 SYSCLK 分频而来。而 SYSCLK 的时钟来源主要包括三个,HSE (high speed extern) 高速外部时钟、HSI (high speed inside) 高速内部时钟、以及 PLL (Phase Locked Loop) 锁相环等。

HSE 高速外部时钟通过在 OSC_IN 和 OSC_OUT 引脚外接一个外部晶振实现。也可以只在 OSC_IN 引脚外接一个现成的时钟源信号。

STM32F4 的内部包含一个内置的 16MHZ 晶振的 HSI 高速内部时钟,它的精度不如外部晶振高,但成本低,启动速度快。可以直接用作 SYSCLK 的时钟输入。下图为 STM32F4 的时钟结构图,可以通过 STM32 的时钟配置工具来配置时钟。

ST 的官方时钟配置工具为 STM32F4xx_Clock_Configuration,是

一个 excel 格式的文件，可以对系统时钟和外设时钟进行配置，并可根据配置直接生成 C 代码，方便使用。

在 STM32 的系统文件中，常常看到 system_stm32f4xx.c 文件，这个文件就是使用工具 STM32F4xx_Clock_Configuration 生成的。

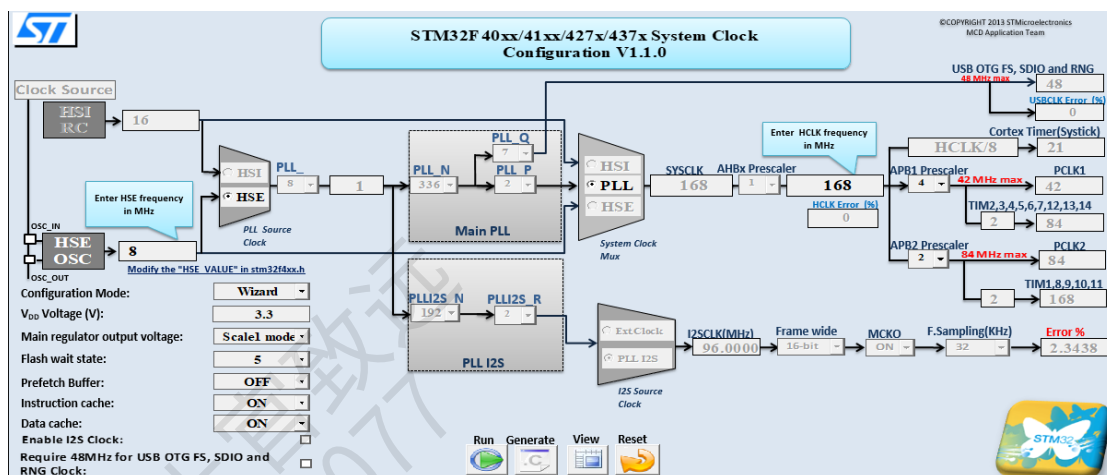


图 7-2 STM32F4 时钟配置

STM32F4 的 PLL 时钟有两个，设计时常用到的是主 PLL。从上图可以看出，它的输入有两个，一个是 HSI，一个是 HSE。常用的外部晶振为 8MHZ，因此，配置时选择 HSE，外部晶振填写 8。

PLL 将 HSE 倍频，以获取更高频率的时钟，给系统各模块使用。上图中的 main PLL 部分是一个倍频器，它的计算公式为：

$$\text{SYSCLK} = \frac{\text{HSE}}{\text{PLL}_M} \times \frac{\text{PLL}_N}{\text{PLL}_P}$$

当 HSE 确定以后，PLL_M、PLL_N 和 PLL_P 的组合有多种，系统时钟的最大频率为 168，常用的设置组合为 PLL_M=8，PLL_N=336，PLL_P=2：

$$\text{SYSCLK} = \frac{8}{8} \times \frac{336}{2} = 168$$

配置完 PLL 得到 SYSCLK 之后，接下来配置总线时钟。STM32 的总线包括 AHB(Advanced High-performance Bus) 和 APB(Advanced Peripheral Bus)。

AHB 称作高级高性能总线，负责 CPU 与 DMA 和 DSP 之间的连接；APB 用于低带宽的外围周边设备之间的连接，比如 USART、ADC、I2C、CAN、TIMER 等设备。

通常情况下 AHBx 的预分频器为 1，即 AHBx Prescaler=1，因此 AHBx 和系统时钟相同。通过 AHBx 预分频得到 APB 时钟及外围模块时钟。

APB1 通过 APB1 Prescaler 得到，最大时钟为 42MHZ，APB2 的最大时钟为 84MHZ，默认以最大时钟配置。

之后可以通过上面的工具，生成 c 文件 system_stm32f4xx.c。该文件有三个函数，SystemInit 用于系统时钟的初始配置，它会调用函数 SetSysClock，里面有 RCC 寄存器的配置以及时钟源选择与预分频的配置等。另外一个函数是 SystemCoreClockUpdate，在时钟配置完成后，调用该函数更新 SystemCoreClock 变量。

值得注意的是在 stm32f4xx.h 中有一个 HSE_VALUE 的变量，需要设置这个变量为自己板子外部晶振的值，常见的为 8M。之后，就可以通过调用 SystemCoreClockUpdate 函数来更新 SystemCoreClock 的值。

系统滴答定时器用于状态机的轮询和系统错误检查任务。建议的设置频率为 2K-4K，可以通过调用 core_cm4.h 里的 SysTick_Config

函数进行设置，它的参数为 `tick`，即两次中断之间的时间间隔计数值，配置时可以这样调用：

```
SysTick_Config(SystemCoreClock / SYS_TICK_FREQUENCY)
```

`SYS_TICK_FREQUENCY` 为定义的滴答定时器的频率，比如为 4K，`SystemCoreClock` 为系统时钟 168M，定时器计数一次的时间为：

$$\frac{168 \times 10^6}{4 \times 10^3} \times \frac{1}{168 \times 10^6} = 250\mu s$$

7.3 PWM 模块配置

PWM 定时器可以采用 STM32F4 的 `TIMER1` 或者 `TIMER8` 这两个高级定时器来产生。高级定时器相比于通用定时器增加了可编程死区、重复计数器以及刹车等功能，特别适合电机的控制。

高级定时器可以产生 3 对 6 路互补的 PWM 波形，并且可以设置死区时间及刹车。从上面的时钟配置原理图中可以看到，高级定时器所在的时钟总线为 `APB2`，经过倍频后，时钟可以高达 168M，与系统时钟同频。

在电机控制中常用到的几个寄存器是 `TIMx_CCMRx` 用于 PWM 模式控制、`TIMx_ARR` 用于 PWM 周期设置、`TIMx_CCRx` 用于占空比设置。一般的配置流程为，首先打开对应的 GPIO 口的时钟，接着打开定时器的时钟，配置 GPIO 口模式，映射定时器对应的 GPIO 端口，最后是定时器寄存器的配置。

以 `TIM8` 为例，电路图中对应的引脚分别如下表所示，在配置时首先打开对应引脚的时钟：

```
/* GPIOA GPIOB GPIOC clocks enable */
RCC_AHB1PeriphClockCmd(RCC_AHB1Periph_GPIOA, ENABLE);
RCC_AHB1PeriphClockCmd(RCC_AHB1Periph_GPIOB, ENABLE);
RCC_AHB1PeriphClockCmd(RCC_AHB1Periph_GPIOC, ENABLE);
```

引脚	功能
PC6	TIM8_CH1
PC7	TIM8_CH2
PC8	TIM8_CH3
PA7	TIM8_CH1N
PB0	TIM8_CH2N
PB1	TIM8_CH3N

表 7-1

之后打开 TIM8 的时钟：

```
/* TIM8 clock enable */
RCC_APB2PeriphClockCmd(RCC_APB2Periph_TIM8, ENABLE);
```

另外还需要对相应的引脚进行功能和输出配置，以 PA7 为例：

```
/* GPIOA Configuration */
GPIO_InitStructure.GPIO_Pin = GPIO_Pin_7 ;
GPIO_InitStructure.GPIO_Mode = GPIO_Mode_AF; //GPIO Alternate function Mode
GPIO_InitStructure.GPIO_Speed = GPIO_Speed_100MHz; //速度
GPIO_InitStructure.GPIO_OType = GPIO_OType_PP; //推挽输出
GPIO_InitStructure.GPIO_PuPd = GPIO_PuPd_NOPULL; //禁用上拉
GPIO_Init(GPIOA, &GPIO_InitStructure);
```

将引脚连接到定时器，也就是映射功能：

```
/* Connect TIM pins to AF1 */
GPIO_PinAFConfig(GPIOA, GPIO_PinSource7, GPIO_AF_TIM8);
```

定时器功能配置：

```
/* Time Base configuration */
TIM_TimeBaseStructure.TIM_Prescaler = PWM_PRSC; //分频系数
TIM_TimeBaseStructure.TIM_CounterMode = TIM_CounterMode_CenterAligned1; //中央对齐模式，从 0 到 TIM_Period，再从 TIM_Period 到 0
TIM_TimeBaseStructure.TIM_Period = PWM_PERIOD; //设置周期
TIM_TimeBaseStructure.TIM_ClockDivision = TIM_CKD_DIV2; //外部时钟滤波系数 手册 p561
```

TIM_TimeBaseStructure.TIM_RepetitionCounter = REP_COUNTER;//重复计数器值的值，就是重复溢出多少次才产生一个溢出中断

```
TIM_TimeBaseInit(TIM8, &TIM_TimeBaseStructure);
```

输出模式：

```
/* Channel 1, 2,3 Configuration in PWM mode */
TIM_OCInitStructure.TIM_OCMode = TIM_OCMode_PWM1;//PWM 模式 1
TIM_OCInitStructure.TIM_OutputState = TIM_OutputState_Enable;//使能 CHx
TIM_OCInitStructure.TIM_OutputNState = TIM_OutputNState_Enable;//使能 CHxN

TIM_OCInitStructure.TIM_OCPolarity = TIM_OCPolarity_High;//高有效
TIM_OCInitStructure.TIM_OCNPolarity = TIM_OCNPolarity_High;
TIM_OCInitStructure.TIM_OCIdleState = TIM_OCIdleState_Reset;//空闲时复位
TIM_OCInitStructure.TIM_OCNIdleState = TIM_OCNIdleState_Reset;
```

配置 CH4 用于触发 ADC 采样：

```
TIM_OCInitStructure.TIM_OutputState = TIM_OutputState_Enable;
TIM_OCInitStructure.TIM_OutputNState = TIM_OutputNState_Disable;
TIM_OCInitStructure.TIM_OCIdleState = TIM_OCIdleState_Reset;
TIM_OCInitStructure.TIM_OCNIdleState = TIM_OCNIdleState_Reset;
TIM_OC4Init(TIM8, &TIM_OCInitStructure);
```

死区设置：

```
TIM_BDTRInitStructure.TIM_DeadTime = DEAD_TIME_COUNTS/2;
```

使能预装载功能：

```
/* Enables the TIM8 Preload on CC1 Register */
TIM_OC1PreloadConfig(TIM8,TIM_OCPreload_Enable);
/* Enables the TIM8 Preload on CC2 Register */
TIM_OC2PreloadConfig(TIM8,TIM_OCPreload_Enable);
/* Enables the TIM8 Preload on CC3 Register */
TIM_OC3PreloadConfig(TIM8,TIM_OCPreload_Enable);
/* Enables the TIM8 Preload on CC4 Register */
TIM_OC4PreloadConfig(TIM8,TIM_OCPreload_Enable);
```

一些变量的定义

```
#define SYSCLK_FREQ                168000000uL
#define PWM_FREQUENCY              16000 //16Khz(N.b.: pattern type is center
aligned)
```

```

/****    Deadtime Value    ****/
#define PWM_DEAD_TIME_NS      800 //800 ns    range is [0...3500]

#define PWM_CYCLES_NS          (u32)(1000000000uL/PWM_FREQUENCY) //62500ns
#define PWM_CYCLES_uS          (u16)(1000000uL/PWM_FREQUENCY)
/****    Pattern type is center aligned    ****/
#define PWM_PRSC                ((u8)0)
/* Resolution: 1Hz */
#define PWM_PERIOD ((u16) (SYSCLK_FREQ / (u32)(2 * PWM_FREQUENCY * (PWM_PRSC+1))))

```

PWM1 模式，即 CNT 小于 CCR 时，输出有效电平，大于 CCR 小于 ARR 时，输出无效电平。

定义 PWM 的频率为 16K，那么

$$\begin{aligned}
 PWM_{PERIOD} &= \frac{SYSCLK_{FREQ}}{2 \times PWM_{FREQUENCY} \times (PWM_{PRSC} + 1)} \\
 &= \frac{168000000}{2 \times 16000 \times 1} = 5250
 \end{aligned}$$

假定某时刻给定 CH1 的 CCR 值为 2625 时，那么选择中央对齐模式且为 PWM1 模式时，波形如下：

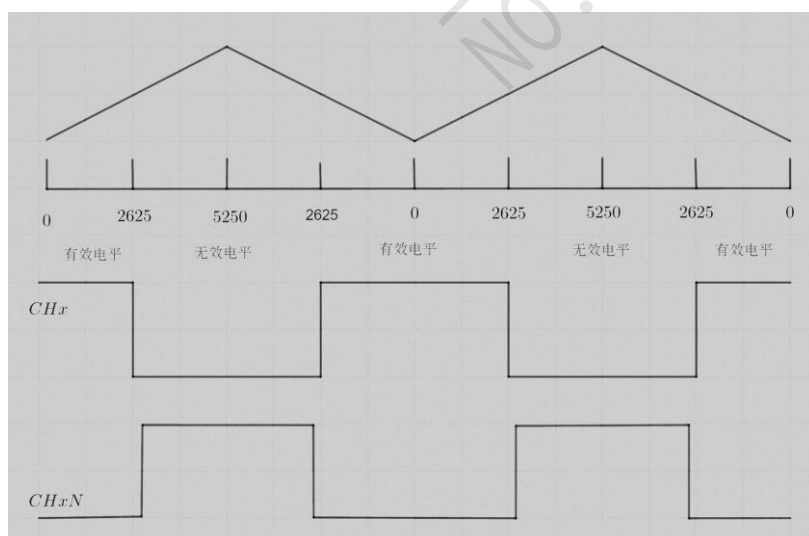


图 7-3 PWM 的中央对齐模式

配置 CH4 通道的目的是为了触发 ADC 采样，根据扇区的位置，可以在 PWM 占空比计算中灵活设置 CH4 的触发点，避开噪声点的位置。

7.4 ADC 模块配置

ADC 模块用于母线电压和电流的采集以及 FOC 关键算法的执行。STM32F4 的 ADC 为 12bit 分辨率，最大值为 4096。和定时器的配置类似，首先打开时钟，然后配置端口功能，端口使用的是 PC0、PC1、PC2，分别对应采集 A、B、C 三路电流。

ADC 对应的总线为 APB2。

```
//APB2 84MHZ
RCC_APB2PeriphClockCmd(RCC_APB2Periph_ADC1, ENABLE);
RCC_AHB1PeriphClockCmd(RCC_AHB1ENR_GPIOCEN, ENABLE);

// Init GPIOC for ADC input
GPIO_InitStruct.GPIO_Mode = GPIO_Mode_AN;//Analog mode.
GPIO_InitStruct.GPIO_Pin = GPIO_Pin_0|GPIO_Pin_1|GPIO_Pin_2;
GPIO_InitStruct.GPIO_PuPd = GPIO_PuPd_NOPULL;//Don't need any pull up or pull down.
GPIO_Init(GPIOC, &GPIO_InitStruct);
...
```

接下来是 ADC 功能配置：

```
ADC_CommonInitStructure.ADC_Mode = ADC_Mode_Independent;//独立模式
ADC_CommonInitStructure.ADC_TwoSamplingDelay = ADC_TwoSamplingDelay_5Cycles;
ADC_CommonInitStructure.ADC_DMAAccessMode = ADC_DMAAccessMode_Disabled;
ADC_CommonInitStructure.ADC_Prescaler = ADC_Prescaler_Div4;//84M/4=21MHZ
ADC_CommonInit(&ADC_CommonInitStructure);
```

```
ADC_InitStruct.ADC_DataAlign = ADC_DataAlign_Right;//右对齐
ADC_InitStruct.ADC_Resolution = ADC_Resolution_12b;//12 位分辨率
```

ADC 的最高频率高达 36M，最快转换时间为 3+12 个周期。参考电压选内部 3.3V。

通常情况下母线电压的采集可以使用规则模式，而电流采集常常使用 ADC 的注入采集模式，注入采集通道能够打断规则通道，优先级较高。

//通道的配置大家可以查阅芯片手册

```
ADC_InjectedChannelConfig(ADC1,ADC_Channel_10,1,ADC_SampleTime_3Cycles);
ADC_InjectedChannelConfig(ADC1,ADC_Channel_11,2,ADC_SampleTime_3Cycles);
ADC_InjectedChannelConfig(ADC1,ADC_Channel_12,3,ADC_SampleTime_3Cycles);
ADC_InjectedSequencerLengthConfig(ADC1,3);//注入通道长度为 3
ADC_ExternalTrigInjectedConvConfig(ADC1,ADC_ExternalTrigInjecConv_T8_CC4);//TIM8_CH4 触发
ADC_ExternalTrigInjectedConvEdgeConfig(ADC1,ADC_ExternalTrigInjecConvEdge_Rising);//上升沿
ADC_AutoInjectedConvCmd(ADC1,DISABLE);//禁用自动注入模式
ADC_ITConfig(ADC1,ADC_IT_JEOC,ENABLE);//打开注入转换完成中断
...
```

这样配置完 ADC 之后，在中断中调用 FOC 核心算法：

```
void ADC_IRQHandler(void)
{

    if(ADC_GetFlagStatus(ADC1,ADC_FLAG_JEOC) == SET)
    {

        /* Clear the ADC1 JEOC pending flag */
        ADC_ClearFlag(ADC1, ADC_FLAG_JEOC);

        switch (BLDC_Type_Motor1.Motor_State)
        {

            case MOTOR_RUN:
                FOC_Core(); //执行 FOC 核心算法
                break;

            case ...
            ...
            break;
            default:
                break;

        }

    }
```

7.5 霍尔接口配置

STM32F4 的 TIM2-TIM5 定时器有一个输入异或功能，支持正交编码器和霍尔传感器。

设计选用的是 TIM5，对应的端口为：

端口	功能
PA0	TIM5_CH1
PA1	TIM5_CH2
PA2	TIM5_CH3

表 7-2

首先使能引脚时钟：

```
RCC_AHB1PeriphClockCmd(RCC_AHB1Periph_GPIOA,ENABLE);
```

引脚配置：

```
GPIO_InitStructure.GPIO_Pin = GPIO_Pin_0|GPIO_Pin_1 |GPIO_Pin_2 ;
GPIO_InitStructure.GPIO_Mode = GPIO_Mode_AF;
GPIO_InitStructure.GPIO_Speed = GPIO_High_Speed;
GPIO_InitStructure.GPIO_PuPd = GPIO_PuPd_NOPULL;
GPIO_Init(GPIOA, &GPIO_InitStructure);
```

//引脚复用映射配置

```
GPIO_PinAFConfig(GPIOA,GPIO_PinSource0,GPIO_AF_TIM5);
GPIO_PinAFConfig(GPIOA,GPIO_PinSource1,GPIO_AF_TIM5);
GPIO_PinAFConfig(GPIOA,GPIO_PinSource2,GPIO_AF_TIM5);
```

接下来是定时器的配置：

```
//tim5 时钟来自 APB1*2 = 42*2 MHZ = 84MHZ
RCC_APB1PeriphClockCmd(RCC_APB1Periph_TIM5, ENABLE);
```

```
// timer base configuration
TIM_TimeBaseStructure.TIM_Prescaler = HALL_TIM_PRESCALER_1M; //时钟频率为 1MHZ
TIM_TimeBaseStructure.TIM_CounterMode = TIM_CounterMode_Up; //增计数模式
TIM_TimeBaseStructure.TIM_Period = HALL_TIM_PERIOD; //周期
TIM_TimeBaseStructure.TIM_ClockDivision = TIM_CKD_DIV1; //分频系数
TIM_TimeBaseStructure.TIM_RepetitionCounter = 0;
TIM_TimeBaseInit(TIM5, &TIM_TimeBaseStructure);
```

使能霍尔接口模式：

```
TIM_SelectHallSensor(TIM5, ENABLE); //使能霍尔输入接口 page 425 TIMx_CR2
TIM_SelectInputTrigger(TIM5, TIM_TS_TI1F_ED); //TI1 边沿检测 page 427 TIMx_SMCR 从模式
控制寄存器
TIM_ICInitStructure.TIM_Channel = TIM_Channel_1;
//霍尔异或中断沿捕获，如果设置为上升或下降，那么每个电周期会有 3 次上升 3 次下降，
那么中断就是 120°电角度
//如果设置为 bothEdge，那么就是 60°电角度中断一次
TIM_ICInitStructure.TIM_ICPolarity = TIM_ICPolarity_BothEdge; //
// listen to T1, the HallSensorEvent
TIM_ICInitStructure.TIM_ICSelection = TIM_ICSelection_TRC;
// Div:1, every edge
TIM_ICInitStructure.TIM_ICPrescaler = TIM_ICPSC_DIV1;
TIM_ICInitStructure.TIM_ICFilter = 15;
TIM_ICInit(TIM5, &TIM_ICInitStructure);
```

打开中断：

```
//CC1 捕获中断，霍尔接口异或配置，有跳变即发生中断
TIM_ITConfig(TIM5, TIM_IT_CC1, ENABLE);
```

之后在定时器捕获中断中根据霍尔的前一个状态和当前状态，进行霍尔电角度的校正，并进行霍尔间隔的计算和存储，用于后续的速度计算。

根据前面的公式，在 ADC 中断中进行实时电角度的计算。通过霍尔转过 60° 扇区的时间，得出角速度。之后根据角速度与 FOC 执行周期做积分运算，得到每个细分的插值位置，用于 FOC 的算法执行。

我们再回顾一下霍尔区间位置估算的算法：

$$\theta_k = \theta_{k-1} + \int_{t_{k-1}}^{t_k} \omega_{k-1} dt$$

在代码实现中，第一步首先是速度 ω_k 的计算，可以采用前面提到的 T 法测速方法。三路霍尔的异或值，每隔 60° 扇区会改变一次，并触发输出一脉冲，使用一个定时器计算这个 60° 扇区的时间，假设这个时间是 T_x ，那么 $\omega_k = \frac{65535}{6 \times T_x}$ ，实际计算中使用 65535 表示 2π 弧度。

时间 dt 为 FOC 的执行周期，一般设置为 1 个 PWM 周期执行一次 FOC，因此 dt 为一个 PWM 的周期。代码中可以这样实现：

```
BLDC_Type_Motor1.DetaEAngle_dpp = 65535/6 * PWM_CYCLES_uS /
(BLDC_Type_Motor1.Hall_Period_Avg)*BLDC_Type_Motor1.Hall_Direction;
```

```
BLDC_Type_Motor1.Hall_EAngle_Cur += BLDC_Type_Motor1.DetaEAngle_dpp;
```

DetaEAngle_dpp 指的是每个 PWM 周期对应的电角度增量。

7.6 编码器接口配置

STM32F4 的定时器 1、2、3、4、5 和 8 都具有编码器接口模式。此次设计，霍尔与编码器使用同一端口，两者只能接其一。不过在伺服控制器的设计中，常常是霍尔与编码器同时使用。霍尔用于初始位置的获取，编码器用于实时位置的获取。两者配合，既能弥补霍尔低分辨率的缺陷，又能弥补增量式编码器无法得到绝对位置的缺点。

STM32 的编码器接口，只支持正交型编码器，只有 A、B 两个输入接口，没有 Z 信号的接口。使用时，可以使用一个 IO 口作为 Z 信号的检测。Z 信号指示编码器的一圈，每隔一圈输出一个脉冲。在转速

很高的时候难免会多输出一个脉冲或少输出一个脉冲，这个时候就会产生误差，那么 Z 信号就可以用来校正计数值。

电机的转向取决于 A 超前于 B 还是 B 超前于 A。

所谓的正交也就是 A 和 B 相差 90° ，因此可以根据 A 和 B 的输出先后顺序，来判断电机旋转的方向。理论上使用 IO 口也可以对编码器计数，但是如果输入的信号有毛刺或干扰，则需要在软件上做一些处理，但是 STM32 的编码器接口具备消除毛刺和干扰的能力，且可以自动识别方向，因此可以很方便的接入编码器。

编码器的输出一般是开漏输出，因此在硬件连接上与霍尔传感器接口的连接类似，常常需要上拉输入。

编码器的配置和霍尔的配置类似，首先是时钟和 GPIO 端口的配置，紧接着是定时器的配置，打开编码器模式等。

以 TIM4 为例，分析一下编码器的配置。

```
RCC_APB1PeriphClockCmd(RCC_APB1Periph_TIM4,ENABLE);
RCC_AHB1PeriphClockCmd(RCC_AHB1Periph_GPIOD,ENABLE);
GPIO_InitStructure.GPIO_Pin = M1_ENCODER_CH1_Pin|M1_ENCODER_CH2_Pin;
GPIO_InitStructure.GPIO_Mode = GPIO_Mode_AF;
GPIO_InitStructure.GPIO_Speed = GPIO_High_Speed;
GPIO_InitStructure.GPIO_PuPd = GPIO_PuPd_NOPULL;
GPIO_Init(M1_ENCODER_GPIO_Port, &GPIO_InitStructure);
RCC_APB1PeriphClockCmd(RCC_APB1Periph_TIM4, ENABLE);
//引脚复用映射配置
GPIO_PinAFConfig(M1_ENCODER_GPIO_Port,GPIO_PinSource12,GPIO_AF_TIM4);
GPIO_PinAFConfig(M1_ENCODER_GPIO_Port,GPIO_PinSource13,GPIO_AF_TIM4);

//定时器设置-----
TIM_TimeBaseInitStructure.TIM_Period = (4 * (M1_ENCODER_PPR)) - 1; //重装载值
TIM_TimeBaseInitStructure.TIM_Prescaler= 0; //预分频
TIM_TimeBaseInitStructure.TIM_CounterMode=TIM_CounterMode_Up; //向上计数
TIM_TimeBaseInitStructure.TIM_ClockDivision=TIM_CKD_DIV1; //时钟分频
```

```
TIM_TimeBaseInit(TIM4,&TIM_TimeBaseInitStructure);//初始化 TIM
```

```
//编码器模式设置-----
```

```
TIM_EncoderInterfaceConfig(TIM4,TIM_EncoderMode_TI12,TIM_ICPolarity_Rising,  
TIM_ICPolarity_Rising);//4 倍频，AB 信号不反向
```

```
TIM_ICInitStructure.TIM_Channel = TIM_Channel_1;  
TIM_ICInitStructure.TIM_ICSelection = TIM_ICSelection_DirectTI; //映射到 TI1 上  
TIM_ICInitStructure.TIM_ICPrescaler = TIM_ICPSC_DIV1;           //配置输入分频,不分频  
TIM_ICInitStructure.TIM_ICFilter = M1_ENC_IC_FILTER;//滤波器值  
TIM_ICInit(TIM4, &TIM_ICInitStructure);  
TIM_ICInitStructure.TIM_Channel = TIM_Channel_2;  
TIM_ICInitStructure.TIM_ICSelection = TIM_ICSelection_DirectTI; //映射到 TI1 上  
TIM_ICInitStructure.TIM_ICPrescaler = TIM_ICPSC_DIV1;           //配置输入分频,不分频  
TIM_ICInitStructure.TIM_ICFilter = M1_ENC_IC_FILTER;//滤波器值  
TIM_ICInit(TIM4, &TIM_ICInitStructure);
```

```
TIM_SelectOutputTrigger(TIM4,TIM_TRGOSource_Reset);
```

```
TIM_SelectMasterSlaveMode(TIM4,TIM_MasterSlaveMode_Disable);
```

```
//溢出中断设置-----
```

```
TIM_ITConfig(TIM4,TIM_IT_Update,ENABLE); //允许 TIM 溢出中断
```

```
//捕获中断
```

```
TIM_ITConfig(TIM4,TIM_IT_CC1,ENABLE); //允许捕获中断
```

编码器的信号 TI1 和 TI2 经过滤波后，边沿检测产生 TI1FP1 以及 TI2FP2。

编码器的模式选择 `TIM_EncoderMode_TI12` 即 TI1 和 TI2 都要计数。

Active edge	Level on opposite signal (TI1FP1 for TI2, TI2FP2 for TI1)	TI1FP1 signal		TI2FP2 signal	
		Rising	Falling	Rising	Falling
Counting on TI1 only	High	Down	Up	No Count	No Count
	Low	Up	Down	No Count	No Count
Counting on TI2 only	High	No Count	No Count	Up	Down
	Low	No Count	No Count	Down	Up
Counting on TI1 and TI2	High	Down	Up	Up	Down
	Low	Up	Down	Down	Up

表 7-3

TI1 和 TI2 对应编码器的 A 和 B 信号，第一列的第三行是就是编

码器的 `TIM_EncoderMode_TI12` 模式，两个都计数。

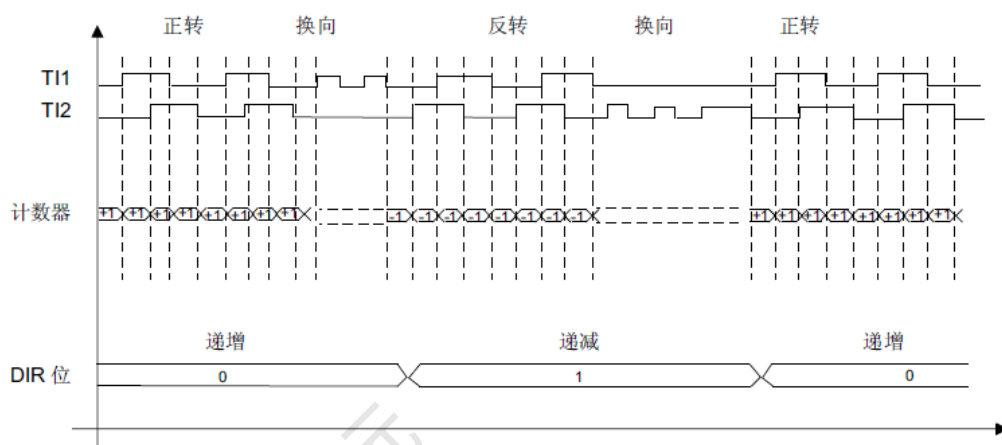


图 7-4 四倍频下的计数模式

4 倍频模式下，TI1 和 TI2 上有电平变化时，定时器的计数器都会进行计数，既可以测速度，也可以测方向，并具有防抖动消除毛刺功能。使用时再配合另外一个定时器计算两个编码器脉冲之间的时间间隔，以便于计算速度。

计数器溢出后，定时器会装载重装载值也就是一圈的脉冲总数，4 倍频下计算方式为：

$$\text{重装载值} = \text{编码器线数} \times 4 - 1$$

重装载后，清零计数，因此可以利用溢出次数获得电机旋转的圈数。

$$\text{转子电角度} = \text{脉冲数} \times 65535 / 4096 \times \text{极对数}$$

可以通过 `TIMx_CR1` 寄存器的 DIR 位获取旋转的方向。

7.7 相电流采集与重构

三电阻采样，在每一个桥臂的下端串联一个电阻，每次只采集其

中两相，根据基尔霍夫电流定律推导出第三相的电流。采样电阻一般选的值很小，也是为了降低采样电阻对逆变桥的影响。

只有在下桥臂导通的情况下，才能检测到有效的电流，根据 SVPWM 的不同扇区决定采集哪两相电流。

每次桥臂开关状态变化时，会在采样电阻上引起电子干扰，假设干扰的时间为 T_{noise} ；下桥臂打开后，需要等待一段时间，等到采样电阻上的电压达到稳定值，假定这段时间为 T_{rise} 。在 T_{noise} 和 T_{rise} 期间不能采集相电流。

由前面的基础矢量圆可知，在扇区 I 和 VI 区间时，任意矢量的合成都需要基础矢量 $U_0(100)$ ，在七段式 PWM 波形中也可以看出来，I 和 VI 扇区的波形都需要基础矢量 $U_0(100)$ 的作用。而且基础矢量 $U_0(100)$ 的作用时间最长。

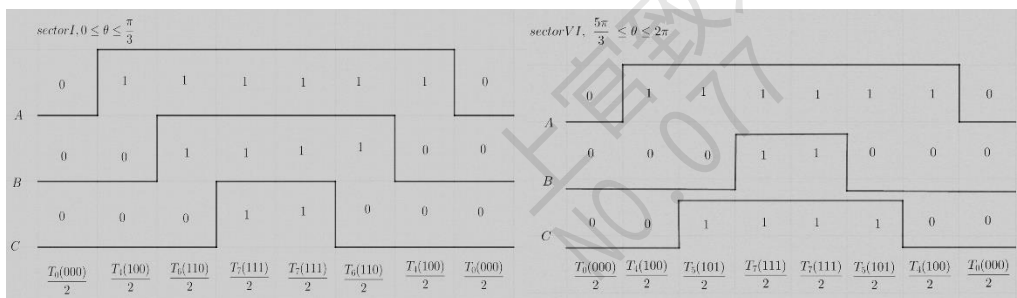


图 7-5 扇区 I 和 VI 的 PWM 波形

因此，A 相上桥臂导通时间最长，下桥臂导通时间最短。因为采集相电流只有在下桥臂导通时才能有效获取，此时可以避开 A 相下桥臂导通太短的缺陷，在三相下桥臂处于导通时只采集 B 和 C 相的电流，由基尔霍夫电流定律推导出 A 相的电流。这就是为什么不同时采集三相的原因。下图的 A、B、C 代表 CHx，对应上桥臂，上桥臂为低，下桥

臂为高。

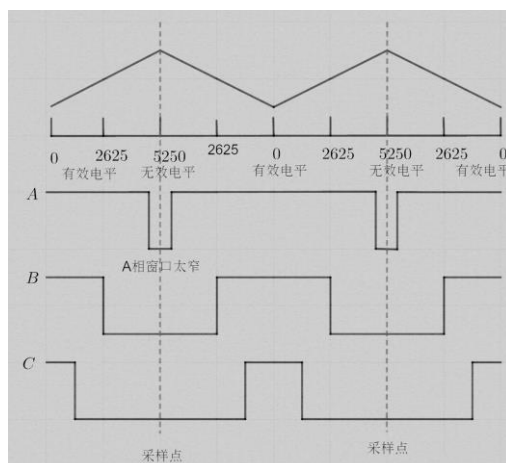


图 7-6 扇区 I 和 VI 采样点

由于电感的作用，电流不会突变，在 A 相上桥臂关闭后，电流会继续原来的方向，由 A 相端线流入电机中性点。此时 B 相的采样电阻和 C 相的采样电阻上的电流流向地，电流会从电源的地流出到 A 相的采样电阻。

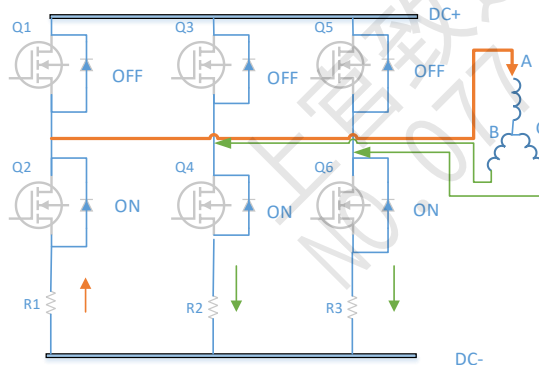


图 7-7 扇区 I 和 VI 相电流

我们假定电流从电机三相端线到中性点的方向为正，反之为负。那么在扇区 I 和 VI 时，电流从 A 相流入，从 B 和 C 相流出，因此 I_a 为正， I_b 和 I_c 为负。

同理在扇区 II 和 III 时都需要基础矢量 $U_{120}(010)$ 的作用，而且 $U_{120}(010)$ 的作用时间最长，因此 B 相的上桥臂导通时间最长，下桥

臂导通时间最短。为了采集到有效的相电流，同样在三相下桥臂同时打开时，只采集 A 相和 C 相的电流。此时电流由 B 相流入，由 A 相和 C 相流出，因此 I_b 为正， I_a 和 I_c 为负。

在扇区 IV 和 V 时都需要基础矢量 $U_{240}(001)$ 的作用，此时 C 相的上桥臂导通时间最长，下桥臂导通时间最短。同样在三相下桥臂都导通时，避开 C 相导通时间短的缺陷，只采集 A 相和 B 相的电流。此时电流由 C 相流入，由 A 相和 B 相流出，因此 I_c 为正， I_a 和 I_b 为负。

三电阻电流采集方案，只需要采集一次便可以重构出三相电流，因此执行速度非常快。

程序中先根据扇区的不同去配置 ADC 应该注入哪两个通道：

```
switch(M1_bSector)
{
case SECTOR_1:
case SECTOR_6:
    ADC_InjectedChannelConfig(ADC1,M1_PHASE_B_ADC_CHANNEL,1,ADC_SampleTime_3Cycles);
    ADC_InjectedChannelConfig(ADC1,M1_PHASE_C_ADC_CHANNEL,2,ADC_SampleTime_3Cycles);
    break;
case SECTOR_2:
case SECTOR_3:
    ADC_InjectedChannelConfig(ADC1,M1_PHASE_A_ADC_CHANNEL,1,ADC_SampleTime_3Cycles);
    ADC_InjectedChannelConfig(ADC1,M1_PHASE_C_ADC_CHANNEL,2,ADC_SampleTime_3Cycles);
    break;
case SECTOR_4:
case SECTOR_5:
    ADC_InjectedChannelConfig(ADC1,M1_PHASE_A_ADC_CHANNEL,1,ADC_SampleTime_3Cycles);
    ADC_InjectedChannelConfig(ADC1,M1_PHASE_B_ADC_CHANNEL,2,ADC_SampleTime_3Cycles);
    break;
}
```

之后，根据当前扇区决定采集哪两相电流：

```
//根据扇区所配置的 ADC 注入通道
hReg1 = ADC_GetInjectedConversionValue(ADC1,ADC_InjectedChannel_1);
hReg2 = ADC_GetInjectedConversionValue(ADC1,ADC_InjectedChannel_2);
switch ( M1_bSector )
```

```

{
case SECTOR_4:
case SECTOR_5: Ia Ib 为负
/* Current on Phase C is not accessible */
/* Ia = PhaseAOffset - ADC converted value) */
wAux = ( int32_t )( BLDC_Type_Motor1.M1_hPhaseAOffset ) - ( int32_t )( hReg1 );
M1_Local_Stator_Currents.qI_Component1 = ( int16_t )wAux;

/* Ib = PhaseBOffset - ADC converted value) */
wAux = ( int32_t )( BLDC_Type_Motor1.M1_hPhaseBOffset ) - ( int32_t )( hReg2 );

M1_Local_Stator_Currents.qI_Component2 = ( int16_t )wAux;
break;

case SECTOR_6:
case SECTOR_1: Ib Ic 为负
/* Current on Phase A is not accessible */
/* Ib = PhaseBOffset - ADC converted value) */
wAux = ( int32_t )( BLDC_Type_Motor1.M1_hPhaseBOffset ) - ( int32_t )( hReg1 );

M1_Local_Stator_Currents.qI_Component2 = ( int16_t )wAux;

/* Ia = -Ic -Ib */
wAux = ( int32_t )( hReg2 ) - ( int32_t )( BLDC_Type_Motor1.M1_hPhaseCOffset ); /* -Ic */
wAux -= ( int32_t )M1_Local_Stator_Currents.qI_Component2; /* Ia */

M1_Local_Stator_Currents.qI_Component1 = ( int16_t )wAux;
break;
case SECTOR_2:
case SECTOR_3: Ia Ic 为负
/* Current on Phase B is not accessible */
/* Ia = PhaseAOffset - ADC converted value) */
wAux = ( int32_t )( BLDC_Type_Motor1.M1_hPhaseAOffset ) - ( int32_t )( hReg1 );

M1_Local_Stator_Currents.qI_Component1 = ( int16_t )wAux;

/* Ib = -Ic -Ia */
wAux = ( int32_t )( hReg2 ) - ( int32_t )( BLDC_Type_Motor1.M1_hPhaseCOffset ); /* -Ic */
wAux -= ( int32_t )M1_Local_Stator_Currents.qI_Component1; /* Ib */
M1_Local_Stator_Currents.qI_Component2 = ( int16_t )wAux;
break;
default:
break;
}

```

7.8 FOC 核心算法

矢量控制的核心算法，是整个无刷电机驱动的关键部分。第三章第五节对矢量控制的整个流程有详细的描述，我们再回顾一下这个流程，

- 首先通过 ADC 采集分流电阻的电压值，获得电流值。
- 之后将采集到的电流值，通过 clarke 与 park 变换，最终得到两相旋转坐标系下的 d 、 q 轴电流。
- 通过传感器获取转子电角度。
- 根据目标速度与实际反馈速度进行比较，送入 PI 控制器进行速度控制，输出电流环需要的 I_q 、 I_d 。
- 根据速度环输出电流值送入电流 PI 控制器最终输出需要的电压值 V_q 、 V_d 。
- 将 d 、 q 轴的电压通过反 park 变换以及 SVPWM，最终形成 PWM 占空比，控制逆变器输出。

整体控制流程图如下：

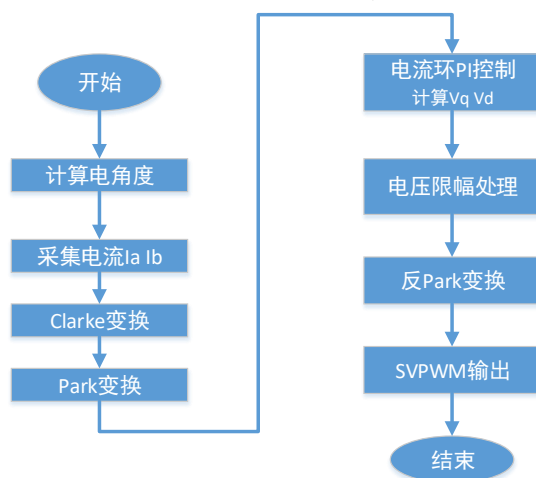


图 7-8 矢量控制核心算法流程

前面已经分析了电角度的计算以及电流的采集，接下来是坐标变换与电流环 PI 控制，这部分不需要什么技巧，只要按公式写代码即可。

流程里有一个环节是电压限幅处理，因为电流环 PI 控制器分别单独输出 v_q 和 v_d ，可以单独控制它们的大小，但是却不能控制它们的合成矢量的大小，因此需要一个限幅的处理。防止 v_q 和 v_d 的合成矢量超过矢量圆。

假设程序里设定的最大电压值为 MODULE_MAX, 在 MCU 中，一般以 S16 格式表示，那么设计时需保证：

$$v_q^2 + v_d^2 \leq \text{MODULE_MAX} \times \text{MODULE_MAX}$$

如果超过该值，则需要等比例的缩小 v_q 和 v_d 的值，缩小的方法见前面合成矢量作用时间的等比例缩小方法。

最后一个环节是 SVPWM 的输出，包括根据电压的关系判断矢量所处的扇区以及占空比的输出。

```
void svpwm_duty_cycle()
{
    s32 U1,U2,U3;
    u8  A,B,C,N;

    s32 K = SQRT_3/U_dc * T_pwm;//
    u16 Tx,Ty;//扇区中两个相邻基础矢量的作用时间
    u16 temp;
    u16 Ta,Tb,Tc;//占空比
    U1 = U_beta;
    U2 = (SQRT_3*U_alpha -U_beta)/2;
    U3 = -(SQRT_3*U_alpha+U_beta)/2;

    if(U1 > 0)
        A = 1;
    else
```

```

    A = 0;

    if(U2 > 0)
        B = 1;
    else
        B = 0;

    if(U3 > 0)
        C = 1;
    else
        C = 0;

    N = A + 2*B+4*C;

    switch(N)
    {
        case 1: Sector = 2; break;
        case 2: Sector = 6; break;
        case 3: Sector = 1; break;
        case 4: Sector = 4; break;
        case 5: Sector = 3; break;
        case 6: Sector = 5; break;
        default: break;
    }

    switch(Sector)
    {
        case 1: Tx =  U2 * K; Ty =  U1 * K;
        case 2: Tx = -U2 * K; Ty = -U3 * K;
        case 3: Tx =  U1 * K; Ty =  U3 * K;
        case 4: Tx = -U1 * K; Ty = -U2 * K;
        case 5: Tx =  U3 * K; Ty =  U2 * K;
        case 6: Tx = -U3 * K; Ty = -U1 * K;
    }

    //过调制的处理
    if(Tx +Ty > T_pwm){
        temp = Tx + Ty;
        Tx = Tx/temp*T_pwm;
        Ty = Ty/temp*T_pwm;
    }

    //根据作用时间计算占空比

```

```

Ta = (T_pwm + Tx + Ty)/4;
Tb = Ta -Tx/2;
Tc = Tb-Ty/2;
//根据扇区给定不同的占空比
switch(Sector){
    case 1:  CCR1 = Ta;CCR2 = Tb;CCR3 = Tc ;   break;
    case 2:  CCR1 = Tb;CCR2 = Ta;CCR3 = Tc;   break;
    case 3:  CCR1 = Tc;CCR2 = Ta;CCR3 = Tb;   break;
    case 4:  CCR1 = Tc;CCR2 = Tb;CCR3 = Ta;   break;
    case 5:  CCR1 = Tb;CCR2 = Tc;CCR3 = Ta;   break;
    case 6:  CCR1 = Ta;CCR2 = Tc;CCR3 = Tb;   break;
    default: break;
}
}

```

7.9 PI 调整方法

评价一个控制系统的性能一般包括三大要素：快、准、稳。动态响应是否够快，控制是否准确，系统是否稳定等。

衡量无刷电机控制系统性能的常见因素包括转矩脉动、启动时间、转矩响应、转速超调、转速响应、转速误差以及启动电流等。电流环和速度环是无刷电机控制系统必要的两个控制环路，两个环路的参数选择是否合适，直接影响无刷电机控制系统的性能。常用的控制方法就是经典的 PID 控制方法。

常规模拟式 PID 以下式给出：

$$u(t) = K_p \left[e(t) + \frac{1}{T_i} \int_0^t e(t) dt + T_d \frac{de(t)}{dt} \right]$$

K_p 为控制器的比例系数， T_i 为控制器的积分时间， T_d 为控制器的微分时间。

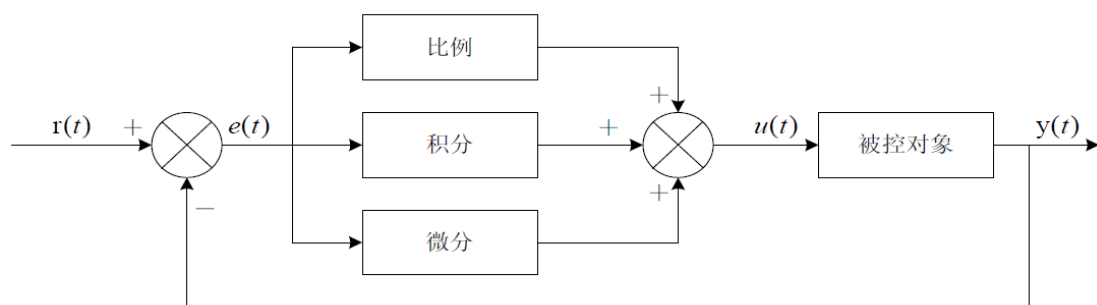


图 7-9 PID 算法

在 MCU 中常常做离散化处理，一种为位置式，一种为增量式。以位置式为例，PID 公式可以离散为：

$$u_k = K_p \times e_k + K_i \sum_{j=0}^k e_j + K_d(e_k - e_{k-1})$$

k 位采样序号， u_k 为第 k 次采样时刻的计算机输出值， e_k 为第 k 次采样时刻输入的偏差值， e_{k-1} 第 k-1 时刻输入的偏差值。

实际中常常使用 PI 控制，于是有了简化的 PI 算法：

$$u_k = K_p \times e_k + K_i \sum_{j=0}^k e_j$$

稍微对 PI 控制器做一些改进，当输出限幅时，不对当前误差进行积分，而是使用上一次的积分值，可以有效的消除稳态误差，避免较大的超调。

当采样周期较小时，且偏差也较小时，积分项很小，在运算时有可能被处理器量化舍弃，从而产生积分量化误差。因此，可以扩大运算变量的字长，提高分辨率和计算精度。但是相应的会带来存储空间的增加以及运算的复杂。但是我们可以只增加积分项的有效字长，并

将它分为整数和尾数两部分，整数与比例部分构成调节器输出，尾数保留下来，作为下一次计算周期的累加的基数值。这样可以有效的减小积分量化误差。

PI 参数常见的调整方法包括试凑法、临界比例法以及解析法。

试凑法较为常见，比较直观，对经验依赖较强。

试凑法的调试流程一般为：

a. 增大比例系数 P 一般会加快系统的响应，在有静差的情况下有利于减小静差，但是过大的比例系数会使系统产生较大的超调，随之带来震荡。

b. 增大积分时间 I ，可以减小静差，有利于减小超调，减小系统震荡，系统稳定性增加；但是过大的积分时间会使系统响应变慢，静差消除时间变长。

c. 增大微分时间 D 有利于加快系统的响应速度，使系统超调量减小，稳定性增加，但是过大的微分时间会使系统对扰动的抑制能力变弱。

电流环处于最内环，应该首先调整电流环，然后将电流环看作速度调节系统中的一个环节，再调试速度环。电流环能够提高系统的快速响应能力，抑制电流环内部干扰，限制最大电流并保障系统安全运行。速度环可以增强系统抗负载扰动的能力，抑制速度突变和波动。

先调整比例环节，将比例系数由小变大，观察系统各次的响应，直到系统反应快、超调小，并使稳态误差足够小。

当稳态误差不能消除时，需要加入积分环节。调整时先将比例系

数减小为原来的 50%到 80%，之后将积分时间给一个较大的值，观测系统的响应曲线，然后减小积分时间，加大比例作用，反复试凑直到达到系统最佳的响应速度和稳定性，确定 PI 参数。

对于电流环系统设计是 d 轴与 q 轴两个 PI 调整系统。调整时可以先将 q 轴 PI 参数设置为 0，整定 d 轴的参数，调整 PI 参数尽量使给定 i_{dref} 跟采集后的电流值经过坐标变换后的 i_d 接近。两个轴的 PI 参数可以使用同一套。

验证时，可以将 i_{qref} 设置为 0，给 i_{dref} 一个值，此时电机不会转，但是用手去转动电机会有有一定的阻力。这就是 d 轴产生的磁场，它可以保持电机处于当前位置。

在控制系统中，将调节器置于纯比例的环节作用下，从小到大逐渐改变调节器的比例系数，将会有有一个等幅震荡的过渡，此时的比例系数称为临界比例系数 K_u ，相邻两个波峰间的时间间隔称为临界震荡周期 T_u 。根据经验公式，计算出调节器的各个参数。调整时遵循先比例后积分再微分的原则。

解析法，需要根据系统的传递函数确定参数，准确度高，能够快速找到系统参数，但是计算复杂，难以理解，不作为本书的重点。前面第三章有对电流环参数计算的一个简单方法，调试时可作为参考。

7.10 有感启动与调速实现

对于带霍尔的无刷电机启动时的方法较为常见的有方波启动再切换到 FOC，另外一种方法就是读取霍尔位置，直接 FOC 启动，虽然霍

尔的精度只有 $\pm 30^\circ$ ，直接给电流启动完全没有问题，但方波启动时力矩更大。

如果系统只接了编码器，启动时则需要预定位。可以给一个 0° 的电压矢量，即 q 轴给 0，d 轴给一个值，由于磁场力的作用，转子最终会转到与给定矢量相同的位置。

对齐之后就可以给电流，根据编码器的脉冲计算速度与电角度，并进行矢量控制。

直流无刷电机调速的根本是调节输入的电压，也就是占空比。根据平均值等效原理，占空比越小，有效电平越窄，作用于电机上的电压越小，最终送到电机上的电流也越小，力矩也越小，换向时间变慢，速度就会降低。电压越大，电流越大，相应的产生的力矩也越大，换向时间加快，从而速度加快。

用给定的速度与反馈速度进行比较，差值送入速度 PI 控制器，输出调整电流，再送入电流 PI 控制器，最后驱动电机，经过速度环不断的调整，最终达到调速的目的。

常见的调速为从基速即额定转速向下调速，还有一种方法是通过降低励磁电流以减弱磁通的调节方式。它是从基速向上调速，称为弱磁控制。由于篇幅有限，弱磁方式不做深入分析。

7.11 矢量控制调试流程

当动手写程序的时候，一开始可以从最小系统写起，先不要追求

太完美，想一次性写好，不太可能。程序都是在不断的调试、增删、修改中慢慢完善的。当然，开始的总体架构要想好，比如程序的整体配置、分几个任务、每个任务之间的调度关系、以及各个中断的处理时刻等，脑子里要有一个大概的框架，这样写起来不至于漫无目的。

当完成系统的基础模块的配置之后，想办法去验证各个模块的功能是否正常。比如 ADC 模块，采集系统母线电压，看采集出来的值经过换算后是否是系统真实值。

验证 ADC 注入通道是否正常触发，确保电流采集功能正常。验证时可以开启定时器的 CH4 通道，看注入通道是否工作。如果正常工作，看采集的值是否是给定的偏置值。

配置高级定时器，输出互补对称的方波，加入死区，改变占空比，示波器查看输出是否合理。测量逆变桥驱动芯片的输出级是否能快速响应输入级的变化，且波形是否符合预期。配合 H 桥驱动，给定几个固定占空比，比如 25%、50%、75%等，万用表测量 H 桥输出是否是电源电压的 $1/4$ 、 $1/2$ 、 $3/4$ ；

调试霍尔传感器时，可以用手拨动电机一圈，在有效停止的位置上看程序是否有中断响应，测试程序响应到的霍尔状态是否按正序或逆序排列。

验证 DAC 功能，给定数字量，用万用表测试输出电压是否是预想值。

当所有的外围基础模块配置验证完毕后，就可以开始进行矢量控制最小系统的调试。

可以先给定一个 v_q ，角度手动累加，如果电机能稳定的旋转起来，那么 SVPWM 算法应该是正确的。这个时候可以使用 DAC 将采集到的电流值输出，并用示波器查看电流波形是否是类正弦。

当电机旋转起来之后，可以开始测试霍尔区间位置估算算法是否正确。使用 DAC 输出电角度，用示波器观察电角度曲线是否是一条平滑上升或下降的直线，也就是 N 形或者倒 N 形波形。如果直线不平滑且台阶明显，那么很可能是同步电角度没有测量正确，或者是积分计算错误。

给定电流值，测试电流环响应是否够快，是否震荡等。

最后加上速度环，测试速度响应曲线；速度的计算一般放到嘀嗒定时器中，速度计算的频率一定要高于速度环的频率，否则调速会不正常。如果有位置环，在确保以上功能正常后再测试位置环。

第八章 常见问题及解决方法

电机开发过程中常常遇到一些意想不到的问题，对于一些常见的部分问题总结如下，仅供参考。

8.1 为什么看不到正弦波

很多初次接触矢量控制的朋友，拿到一块 Demo 板，迫不及待的想看看所谓的矢量控制的正弦波形，但是很遗憾，用示波器测了半天发现根本没有正弦波。于是怀疑，甚至致电客服：您这板子是不是发错了，上面跑的是矢量控制程序吗，怎么看不到正弦波呢？

为了说明白这个问题，先搞清楚几个名词：端电压 相电压 线电压。

端电压指的是电机端线对地的电压，通过在电机端接一个 RC 低通滤波器可以看到马鞍波。

相电压指的是电机端线到中性点的电压，理论上是正弦波，但是示波器没办法测到，但是我们可以测相电流， $I = U/Z$ ，相电流是正弦，相电压也是正弦。

线电压是电机任意两个端线之间的电压，理论上它是正弦的，示波器测量也是看不到的，示波器看到的是载波信号，必须经过 RC 滤波后才能看到正弦。

限于篇幅，SVPWM 马鞍波产生的原因本书不做论述，我们会在下一

版中添加该部分的详细推导。

8.2 速度波动为什么那么大

调试速度闭环时，常常发现速度的波动很大，高速时稍微平稳一些，特别是在低速段，有时候电机根本无法正常运行。

速度的精确测量一直是无刷电机驱动技术中比较关键的一个环节。可以采用 M/T 相结合的方法，来测量速度。另外可以对速度做一些平滑处理，加入滑动滤波算法。

滑动滤波算法的基本原理是，将每次采样的值都放入一个队列的尾部，将此次采样的值和前几次采样的值放在一起求平均。程序设计时可以使用环形队列的方式实现，每次放进去一个新值，同时去掉一个老值。滑动滤波可以有效的去除一些速度计算的干扰，提高速度计算的精度。

8.3 如何选择电机的最大转矩

电机的最大转矩必须能涵盖负载转矩 T_L 、惯性转矩 T_J 以及克服摩擦所需的转矩 T_F ，考虑最大转矩除满足以上各种转矩之外还需要额外考虑一些其它情况的影响，包括风阻损失、以及现场复杂工况的影响等因素。所以，在选择电机的最大转矩时，应该考虑一定的富余量。一般会留出 20% 的安全裕度。

$$T_P = (T_L + T_J + T_F) \times (1 + 20\%)$$

惯性转矩是将负载从静止加速到高速所需的转矩，通常用下面的

公式计算：

$$T_J = (J_L + M) \times \alpha$$

M 为负载， α 为加速度。

8.4 电机运行时为什么会发烫

有时候开发过程中，电机运行起来了，看相电流也比较正弦，但是电机就是发烫。电机发烫往往是效率转换低的原因。电机是电能转化成机械能的装置，内部由线圈组成，如果电机持续发烫，很可能是电能并没有很好的转化成动能，而是消耗在线圈的发热上。

当电角度估算不正确时，电机有一种被拖动的感觉，噪声也大，此时电机运行效率降低，有可能引起电机发烫。当然电机运行过程中会有部分转化成热能，但正常情况下不至于发烫。

8.5 如何判断电机的噪声来源

开发过程中经常会遇到电机噪声的现象，因为电机是一个动态复杂的系统，很难直观的确定噪声的来源。如果是电磁噪声，它会随着磁场强弱以及负载电流大小而改变，可以将电机空载运行一段时间后，突然切断电源，如果有部分噪声随之消失，那么这部分噪声就是电磁噪声；剩下的噪声就是机械噪声。

测试三相电流是否对称，如果有断相或者定子绕线短路的情况下，电流由于不对称会产生一定的噪声。

另外，当角度估算不准确时，电机效率下降，电机运行不平稳，

会有一定的噪声存在。

气隙磁场中除了基波分量外，还存在高次谐波，高次谐波的径向力波分别作用于转子和定子铁芯上，会引起它们的径向变形以及周期震动，定子变形后会引发周边空气震动，形成一个声辐射器，从而产生噪声。

上官致远
No.077

上官致远
No.077

参考文献

- [1] 唐任远. 现代永磁电机理论与设计机械工业出版社, 2005
- [2] 汤蕴璆等. 电机学 第三版, 机械工业出版社, 2008
- [3] 刘刚等. 永磁无刷直流电机控制技术与应用, 机械工业出版社, 2008
- [4] 邱国平. 永磁直流电机实用设计及应用设计, 机械工业出版社, 2009
- [5] 胡岩等. 小型电动机现代实用设计技术, 机械工业出版社, 2008
- [6] 王秀和等. 永磁电机, 中国电力出版社, 2007
- [7] 汤蕴璆等. 交流电机动态分析, 机械工业出版社, 2005
- [8] 王成元等. 电机现代控制技术, 机械工业出版社, 2006
- [9] 陈世坤. 电机设计, 机械工业出版社
- [10] 谭建成. 永磁无刷直流电机技术, 机械工业出版社, 2011
- [11] 陈伯时等. 电力拖动自动控制系统-运动控制系统, 第五版, 2016
- [12] [日]江崎雅康. 无刷电机矢量控制技术, 科学技术出版社, 2018
- [13] 袁雷. 现代永磁同步电机控制原理及 MATLAB 仿真, 北京航空航天大学出版社, 2016
- [14] 夏长亮. 无刷直流电机控制系统, 科学出版社, 2009
- [15] 陈炜. 永磁无刷直流电机换相转矩脉动抑制技术研究. 天津大学博士学位论文, 2006
- [16] 宋海龙. 无刷直流电动机空调控制系统研究, 哈尔滨工业大学博士论文, 2003
- [17] 曾允文. 变频调速 SVPWM 技术的原理与应用
- [18] MicroChip. AN885, 无刷直流电机基础
- [19] TI. ZHCA554, 三相永磁同步电机的传感器磁场定向控制
- [20] MicroChip. AN1078 PMSM 的无传感器磁场定向控制
- [21] James L. Kirtley Jr. Massachusetts Institute of Technology Department of Electrical Engineering and Computer Science.
- [22] David G. Luenberger. Optimization by Vector Space Methods
- [23] 周渊深. 交直流调速系统与 MATLAB 仿真, 中国电力出版社, 2003
- [24] 王晓明. 电动机的 DSP 控制-TI 公司 DSP 应用, 第二版
- [25] 许家群等. 电动汽车驱动用永磁同步电动机系统效率优化控制研究
- [26] 宋建国. 基于标幺化模型的 PMSM 矢量控制系统设计
- [27] 李阳. 永磁同步电机矢量控制系统的标幺化设计
- [28] 徐德鸿. 电力电子系统建模及控制, 机械工业出版社 2011
- [29] 袁登科. 永磁同步电机变频调速系统及其控制, 机械工业出版社, 2015
- [30] 张兴. PWM 整流器及其控制, 机械工业出版社, 2012